

Assignment 2 --- Fraud Detection

1. Import Library

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# split data into train & test
from sklearn.model_selection import train_test_split

# Disable Warnings
import warnings
warnings.filterwarnings('ignore')
```

1. Read DataSet

```
In [17]: url = "https://raw.githubusercontent.com/ammishra08/Machine-
Learning/refs/heads/master/Datasets/fraud_oracle.csv"

df1 = pd.read_csv(url)

print("Dataset loaded successfully!")
```

Dataset loaded successfully!

1. Displaying First 10 Rows

```
In [18]: df1.head(10)
```

Out[18]: Columns 1–8 of 22

	Month	WeekOfMonth	DayOfWeek	Make	AccidentArea	DayOfWeekClaimed	MonthClaimed
0	Dec	5	Wednesday	Honda	Urban	Tuesday	Jan
1	Jan	3	Wednesday	Honda	Urban	Monday	Jan
2	Oct	5	Friday	Honda	Urban	Thursday	Nov
3	Jun	2	Saturday	Toyota	Rural	Friday	Jul
4	Jan	5	Monday	Honda	Urban	Tuesday	Feb
5	Oct	4	Friday	Honda	Urban	Wednesday	Nov
6	Feb	1	Saturday	Honda	Urban	Monday	Feb
7	Nov	1	Friday	Honda	Urban	Tuesday	Mar
8	Dec	4	Saturday	Honda	Urban	Wednesday	Dec
9	Apr	3	Tuesday	Ford	Urban	Wednesday	Apr

Columns 9–15 of 22 · first column repeated

	WeekOfMonthClaimed	Sex	MaritalStatus	...	AgeOfVehicle	AgeOfPolicyHolder	PoliceReportFiled
0	1	Female	Single	...	3 years	26 to 30	No
1	4	Male	Single	...	6 years	31 to 35	Yes
2	2	Male	Married	...	7 years	41 to 50	No
3	1	Male	Married	...	more than 7	51 to 65	Yes
4	2	Female	Single	...	5 years	31 to 35	No
5	1	Male	Single	...	5 years	21 to 25	No
6	3	Male	Married	...	7 years	36 to 40	No
7	4	Male	Single	...	new	16 to 17	No
8	5	Male	Single	...	6 years	31 to 35	No
9	3	Male	Married	...	more than 7	36 to 40	No

Columns 16–22 of 22 · first column repeated

	WitnessPresent	AgentType	NumberOfSuppliments	AddressChange_Claim	NumberOfCars	Year	BasePolicy
0	No	External	none	1 year	3 to 4	1994	Liability
1	No	External	none	no change	1 vehicle	1994	Collision
2	No	External	none	no change	1 vehicle	1994	Collision
3	No	External	more than 5	no change	1 vehicle	1994	Liability
4	No	External	none	no change	1 vehicle	1994	Collision
5	No	External	3 to 5	no change	1 vehicle	1994	Collision
6	No	External	1 to 2	no change	1 vehicle	1994	Collision
7	No	External	none	no change	1 vehicle	1994	Collision
8	Yes	External	3 to 5	no change	1 vehicle	1994	Collision
9	No	External	3 to 5	no change	1 vehicle	1994	All Perils

10 rows × 33 columns

```
In [ ]: print("Dataset shape:", df1.shape)
```

Dataset shape: (15420, 33)

```
In [ ]: df1.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 15420 entries, 0 to 15419
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Month                                15420 non-null  str
1   WeekOfMonth                          15420 non-null  int64
2   DayOfWeek                            15420 non-null  str
3   Make                                 15420 non-null  str
4   AccidentArea                        15420 non-null  str
5   DayOfWeekClaimed                    15420 non-null  str
6   MonthClaimed                        15420 non-null  str
7   WeekOfMonthClaimed                  15420 non-null  int64
8   Sex                                  15420 non-null  str
9   MaritalStatus                       15420 non-null  str
10  Age                                  15420 non-null  int64
11  Fault                                15420 non-null  str
12  PolicyType                           15420 non-null  str
13  VehicleCategory                      15420 non-null  str
14  VehiclePrice                         15420 non-null  str
15  FraudFound_P                         15420 non-null  int64
16  PolicyNumber                         15420 non-null  int64
17  RepNumber                            15420 non-null  int64
18  Deductible                           15420 non-null  int64
19  DriverRating                         15420 non-null  int64
20  Days_Policy_Accident                 15420 non-null  str
21  Days_Policy_Claim                    15420 non-null  str
22  PastNumberOfClaims                   15420 non-null  str
23  AgeOfVehicle                         15420 non-null  str
24  AgeOfPolicyHolder                    15420 non-null  str
25  PoliceReportFiled                    15420 non-null  str
26  WitnessPresent                       15420 non-null  str
27  AgentType                            15420 non-null  str
28  NumberOfSuppliments                  15420 non-null  str
29  AddressChange_Claim                  15420 non-null  str
30  NumberOfCars                         15420 non-null  str
31  Year                                  15420 non-null  int64
32  BasePolicy                           15420 non-null  str
dtypes: int64(9), str(24)
memory usage: 3.9 MB
```

1. Check And Handling Missing Values

```
In [ ]: df1.isnull().sum()
```

```
Out[6]: Month                                0
WeekOfMonth                                0
DayOfWeek                                  0
Make                                        0
AccidentArea                              0
DayOfWeekClaimed                          0
MonthClaimed                              0
WeekOfMonthClaimed                        0
Sex                                         0
MaritalStatus                             0
Age                                         0
Fault                                      0
PolicyType                                0
VehicleCategory                           0
VehiclePrice                              0
FraudFound_P                              0
PolicyNumber                              0
RepNumber                                  0
Deductible                                0
DriverRating                              0
Days_Policy_Accident                      0
Days_Policy_Claim                         0
PastNumberOfClaims                        0
AgeOfVehicle                              0
AgeOfPolicyHolder                         0
PoliceReportFiled                         0
WitnessPresent                            0
AgentType                                  0
NumberOfSupplements                       0
AddressChange_Claim                       0
NumberOfCars                              0
Year                                       0
BasePolicy                                0
dtype: int64
```

```
In [19]: # show descriptive summary of sample data
df1.describe()
```

Out[19]: Columns 1–8 of 10

	WeekOfMonth	WeekOfMonthClaimed	Age	FraudFound_P	PolicyNumber	RepNumber	Deductible
count	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000
mean	2.788586	2.693969	39.855707	0.059857	7710.500000	8.483268	407.704280
std	1.287585	1.259115	13.492377	0.237230	4451.514911	4.599948	43.950998
min	1.000000	1.000000	0.000000	0.000000	1.000000	1.000000	300.000000
25%	2.000000	2.000000	31.000000	0.000000	3855.750000	5.000000	400.000000
50%	3.000000	3.000000	38.000000	0.000000	7710.500000	8.000000	400.000000
75%	4.000000	4.000000	48.000000	0.000000	11565.250000	12.000000	400.000000
max	5.000000	5.000000	80.000000	1.000000	15420.000000	16.000000	700.000000

Columns 9–10 of 10 · first column repeated

	DriverRating	Year
count	15420.000000	15420.000000
mean	2.487808	1994.866472
std	1.119453	0.803313
min	1.000000	1994.000000
25%	1.000000	1994.000000
50%	2.000000	1995.000000
75%	3.000000	1996.000000
max	4.000000	1996.000000

```
In [ ]: df1.describe(include = 'str')
```

Out[9]: Columns 1–8 of 22

	Month	DayOfWeek	Make	AccidentArea	DayOfWeekClaimed	MonthClaimed	Sex
count	15420	15420	15420	15420	15420	15420	15420
unique	12	7	19	2	8	13	2
top	Jan	Monday	Pontiac	Urban	Monday	Jan	Male
freq	1411	2616	3837	13822	3757	1446	13000

Columns 9–15 of 22 · first column repeated

	MaritalStatus	Fault	PolicyType	...	PastNumberOfClaims	AgeOfVehicle	AgeOfPolicyHolder
count	15420	15420	15420	...	15420	15420	15420
unique	4	2	9	...	4	8	9
top	Married	Policy Holder	Sedan - Collision	...	2 to 4	7 years	31 to 35
freq	10625	11230	5584	...	5485	5807	5593

Columns 16–22 of 22 · first column repeated

	PoliceReportFiled	WitnessPresent	AgentType	NumberOfSupplements	AddressChange_Claim	NumberOfCars	BasePolicy
count	15420	15420	15420	15420	15420	15420	15420
unique	2	2	2	4	5	5	3
top	No	No	External	none	no change	1 vehicle	Collision
freq	14992	15333	15179	7047	14324	14316	5962

4 rows × 24 columns

```
In [35]: df1['DayOfWeekClaimed'].unique()
```

```
Out[35]: <StringArray>
[ 'Tuesday',    'Monday',  'Thursday',    'Friday', 'Wednesday',
  'Saturday',
    'Sunday',      '0']
Length: 8, dtype: str
```

```
In [36]: df1['MonthClaimed'].unique()
```

```
Out[36]: <StringArray>
['Jan', 'Nov', 'Jul', 'Feb', 'Mar', 'Dec', 'Apr', 'Aug', 'May', 'Jun',
  'Sep',
  'Oct',  '0']
Length: 13, dtype: str
```

```
In [ ]: df1['Age'].unique()
```

```
Out[20]: array([0])
```

```
In [21]: # Replacing 0 in Age column with the mean of non-zero values
df1['Age'] = df1['Age'].replace(0, df1.loc[df1['Age'] != 0, 'Age'].mean())
```

```
In [23]: df1.describe()
```

Out[23]: Columns 1–8 of 10

	WeekOfMonth	WeekOfMonthClaimed	Age	FraudFound_P	PolicyNumber	RepNumber	Deductible
count	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000
mean	2.788586	2.693969	40.700331	0.059857	7710.500000	8.483268	407.704280
std	1.287585	1.259115	12.181090	0.237230	4451.514911	4.599948	43.950998
min	1.000000	1.000000	16.000000	0.000000	1.000000	1.000000	300.000000
25%	2.000000	2.000000	31.000000	0.000000	3855.750000	5.000000	400.000000
50%	3.000000	3.000000	39.000000	0.000000	7710.500000	8.000000	400.000000
75%	4.000000	4.000000	48.000000	0.000000	11565.250000	12.000000	400.000000
max	5.000000	5.000000	80.000000	1.000000	15420.000000	16.000000	700.000000

Columns 9–10 of 10 · first column repeated

	DriverRating	Year
count	15420.000000	15420.000000
mean	2.487808	1994.866472
std	1.119453	0.803313
min	1.000000	1994.000000
25%	1.000000	1994.000000
50%	2.000000	1995.000000
75%	3.000000	1996.000000
max	4.000000	1996.000000

```
In [24]: print("Number of duplicate rows:", df.duplicated().sum())
```

Number of duplicate rows: 0

1. Separate numerical and categorical variables

```
In [27]: numeric_columns = df1.select_dtypes(include=np.number).columns.tolist()
categorical_columns = df1.select_dtypes(include='object').columns.tolist()

print("Numerical Columns:")
print(numeric_columns)

print("\nCategorical Columns:")
print(categorical_columns)

Numerical Columns:
['WeekOfMonth', 'WeekOfMonthClaimed', 'Age', 'FraudFound_P', 'PolicyNumber',
'RepNumber', 'Deductible', 'DriverRating', 'Year']

Categorical Columns:
['Month', 'DayOfWeek', 'Make', 'AccidentArea', 'DayOfWeekClaimed',
'MonthClaimed', 'Sex', 'MaritalStatus', 'Fault', 'PolicyType',
'VehicleCategory', 'VehiclePrice', 'Days_Policy_Accident',
'Days_Policy_Claim', 'PastNumberOfClaims', 'AgeOfVehicle',
'AgeOfPolicyHolder', 'PoliceReportFiled', 'WitnessPresent', 'AgentType',
'NumberOfSupplements', 'AddressChange_Claim', 'NumberOfCars', 'BasePolicy']
```

1. Exploratory Data Analysis

```
In [28]: df1[numeric_columns].describe()
```

Out[28]: Columns 1–8 of 10

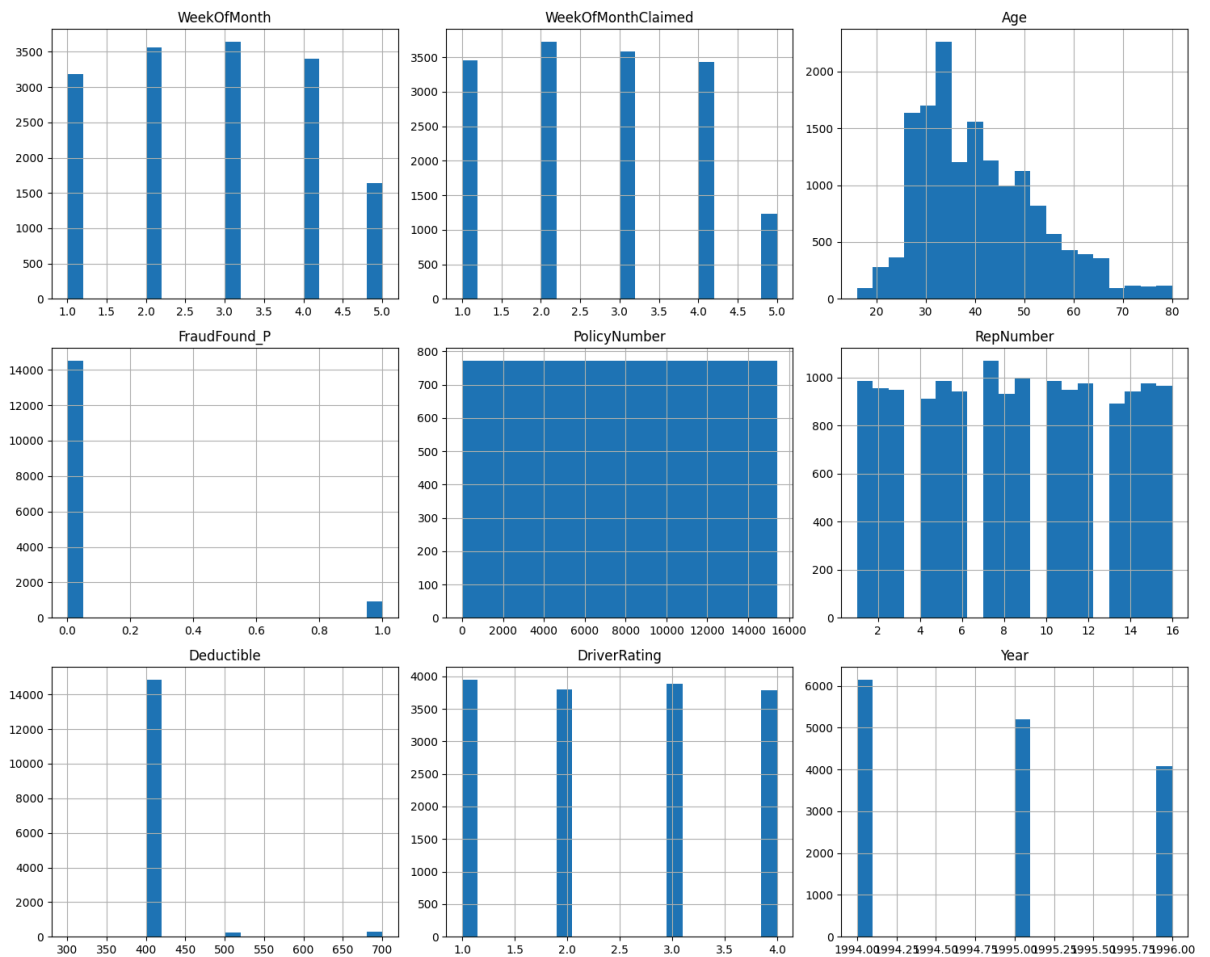
	WeekOfMonth	WeekOfMonthClaimed	Age	FraudFound_P	PolicyNumber	RepNumber	Deductible
count	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000	15420.000000
mean	2.788586	2.693969	40.700331	0.059857	7710.500000	8.483268	407.704280
std	1.287585	1.259115	12.181090	0.237230	4451.514911	4.599948	43.950998
min	1.000000	1.000000	16.000000	0.000000	1.000000	1.000000	300.000000
25%	2.000000	2.000000	31.000000	0.000000	3855.750000	5.000000	400.000000
50%	3.000000	3.000000	39.000000	0.000000	7710.500000	8.000000	400.000000
75%	4.000000	4.000000	48.000000	0.000000	11565.250000	12.000000	400.000000
max	5.000000	5.000000	80.000000	1.000000	15420.000000	16.000000	700.000000

Columns 9–10 of 10 · first column repeated

	DriverRating	Year
count	15420.000000	15420.000000
mean	2.487808	1994.866472
std	1.119453	0.803313
min	1.000000	1994.000000
25%	1.000000	1994.000000
50%	2.000000	1995.000000
75%	3.000000	1996.000000
max	4.000000	1996.000000

```
In [29]: df1[numeric_columns].hist(
          figsize=(15, 12),
          bins=20
        )

plt.tight_layout()
plt.show()
```



```
In [31]: for column in categorical_columns:
          print("\n", column)
          print(df1[column].unique())
```

```
Month
<StringArray>
['Dec', 'Jan', 'Oct', 'Jun', 'Feb', 'Nov', 'Apr', 'Mar', 'Aug', 'Jul',
 'May',
 'Sep']
Length: 12, dtype: str
```

```
DayOfWeek
<StringArray>
['Wednesday', 'Friday', 'Saturday', 'Monday', 'Tuesday', 'Sunday',
 'Thursday']
Length: 7, dtype: str
```

```
Make
<StringArray>
[ 'Honda',    'Toyota',    'Ford',    'Mazda', 'Chevrolet',
 'Pontiac',
  'Accura',    'Dodge',    'Mercury',    'Jaguar',    'Nissan',
 'VW',
  'Saab',    'Saturn',    'Porche',    'BMW',    'Mecedes',
 'Ferrari',
  'Lexus']
Length: 19, dtype: str
```

```
AccidentArea
<StringArray>
['Urban', 'Rural']
Length: 2, dtype: str
```

```
DayOfWeekClaimed
<StringArray>
[ 'Tuesday',    'Monday',    'Thursday',    'Friday', 'Wednesday',
 'Saturday',
  'Sunday',    '0']
Length: 8, dtype: str
```

```
MonthClaimed
<StringArray>
['Jan', 'Nov', 'Jul', 'Feb', 'Mar', 'Dec', 'Apr', 'Aug', 'May', 'Jun',
 'Sep',
 'Oct',    '0']
Length: 13, dtype: str
```

```
Sex
<StringArray>
['Female', 'Male']
Length: 2, dtype: str
```

```
MaritalStatus
```

```
<StringArray>
['Single', 'Married', 'Widow', 'Divorced']
Length: 4, dtype: str
```

```
Fault
<StringArray>
['Policy Holder', 'Third Party']
Length: 2, dtype: str
```

```
PolicyType
<StringArray>
[ 'Sport - Liability',      'Sport - Collision',      'Sedan - Liability',
  'Utility - All Perils',   'Sedan - All Perils',     'Sedan - Collision',
  'Utility - Collision',    'Utility - Liability',    'Sport - All Perils']
Length: 9, dtype: str
```

```
VehicleCategory
<StringArray>
['Sport', 'Utility', 'Sedan']
Length: 3, dtype: str
```

```
VehiclePrice
<StringArray>
['more than 69000', '20000 to 29000', '30000 to 39000', 'less than 20000',
 '40000 to 59000', '60000 to 69000']
Length: 6, dtype: str
```

```
Days_Policy_Accident
<StringArray>
['more than 30', '15 to 30', 'none', '1 to 7', '8 to 15']
Length: 5, dtype: str
```

```
Days_Policy_Claim
<StringArray>
['more than 30', '15 to 30', '8 to 15', 'none']
Length: 4, dtype: str
```

```
PastNumberOfClaims
<StringArray>
['none', '1', '2 to 4', 'more than 4']
Length: 4, dtype: str
```

```
AgeOfVehicle
<StringArray>
[ '3 years',      '6 years',      '7 years', 'more than 7',      '5 years',
  'new',          '4 years',      '2 years']
Length: 8, dtype: str
```

```
AgeOfPolicyHolder
<StringArray>
['26 to 30', '31 to 35', '41 to 50', '51 to 65', '21 to 25', '36 to 40',
 '16 to 17', 'over 65', '18 to 20']
Length: 9, dtype: str
```

PoliceReportFiled
<StringArray>
['No', 'Yes']
Length: 2, dtype: str

WitnessPresent
<StringArray>
['No', 'Yes']
Length: 2, dtype: str

AgentType
<StringArray>
['External', 'Internal']
Length: 2, dtype: str

NumberOfSuppliments
<StringArray>
['none', 'more than 5', '3 to 5', '1 to 2']
Length: 4, dtype: str

AddressChange_Claim
<StringArray>
['1 year', 'no change', '4 to 8 years', '2 to 3 years', 'under 6 months']
Length: 5, dtype: str

NumberOfCars
<StringArray>
['3 to 4', '1 vehicle', '2 vehicles', '5 to 8', 'more than 8']
Length: 5, dtype: str

BasePolicy
<StringArray>
['Liability', 'Collision', 'All Perils']
Length: 3, dtype: str

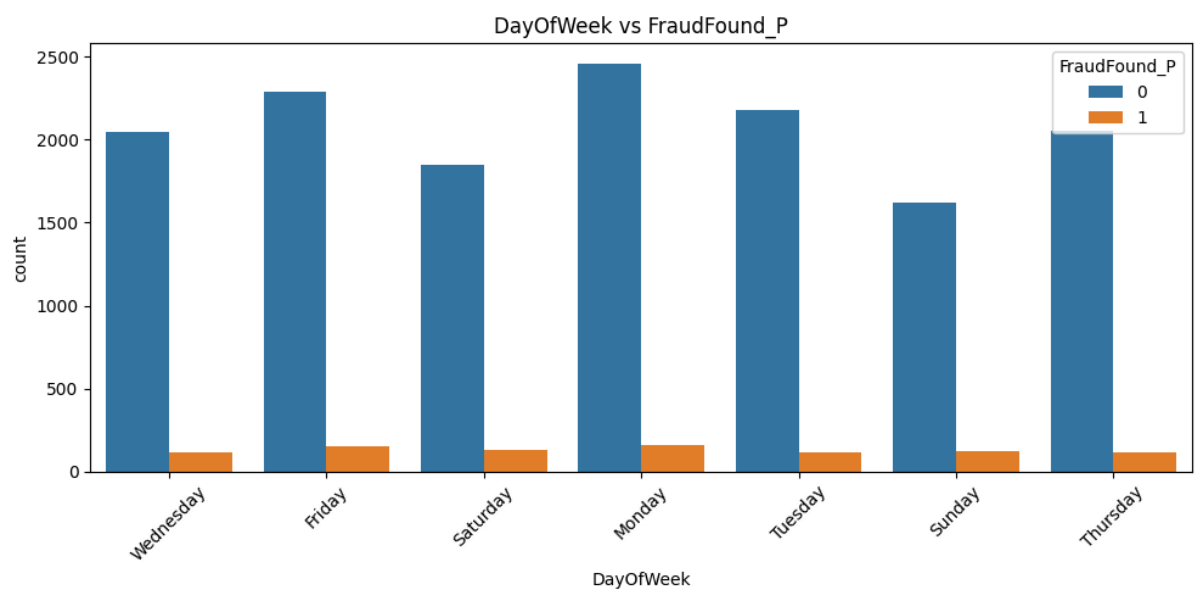
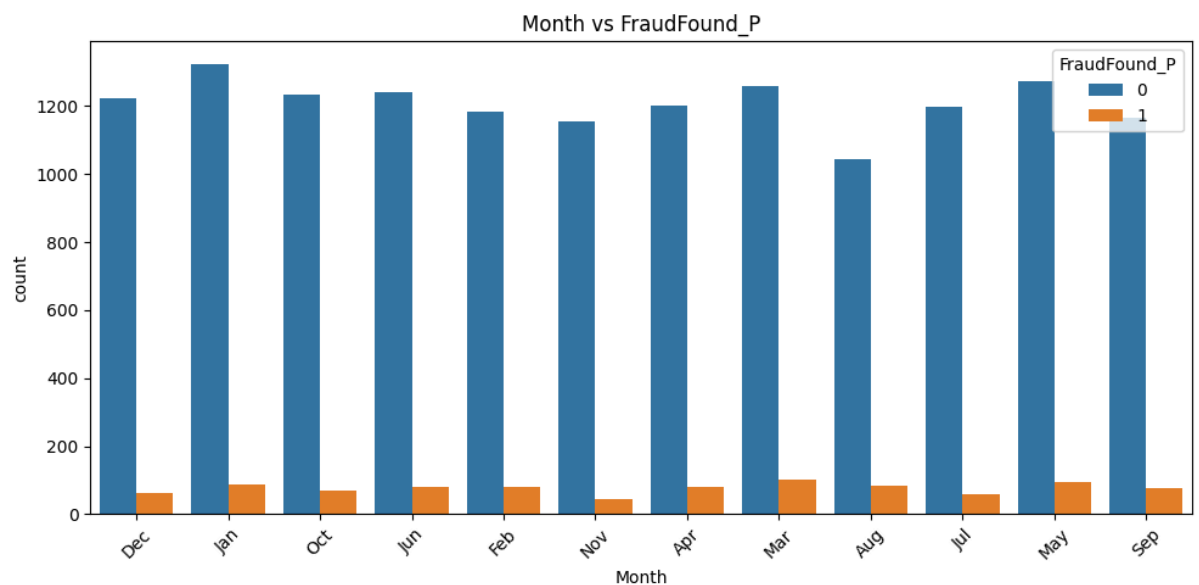
```
In [32]: for column in categorical_columns:

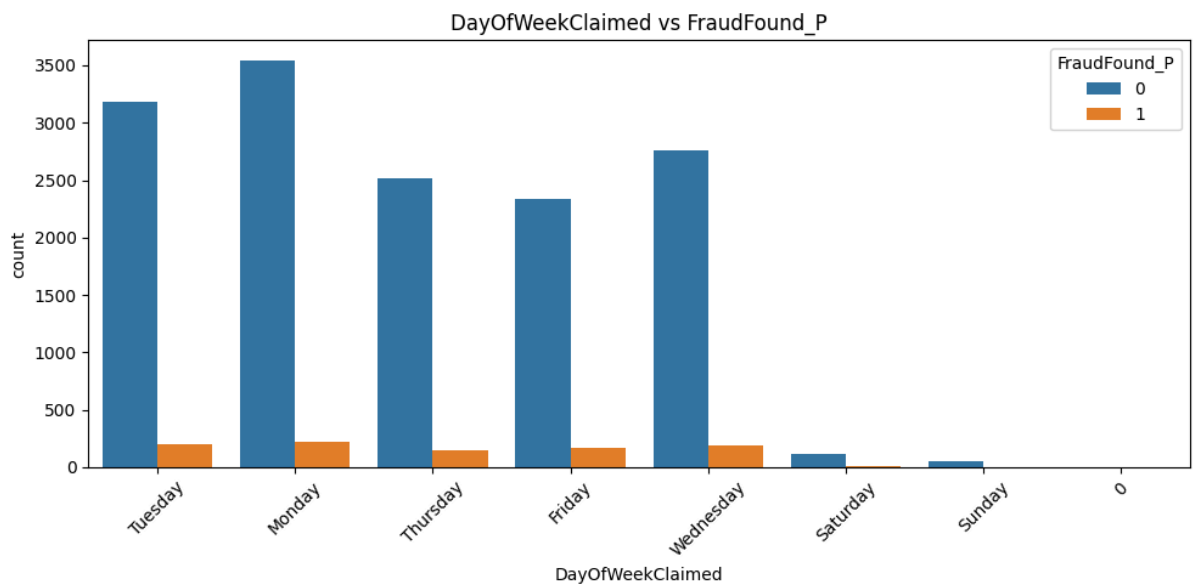
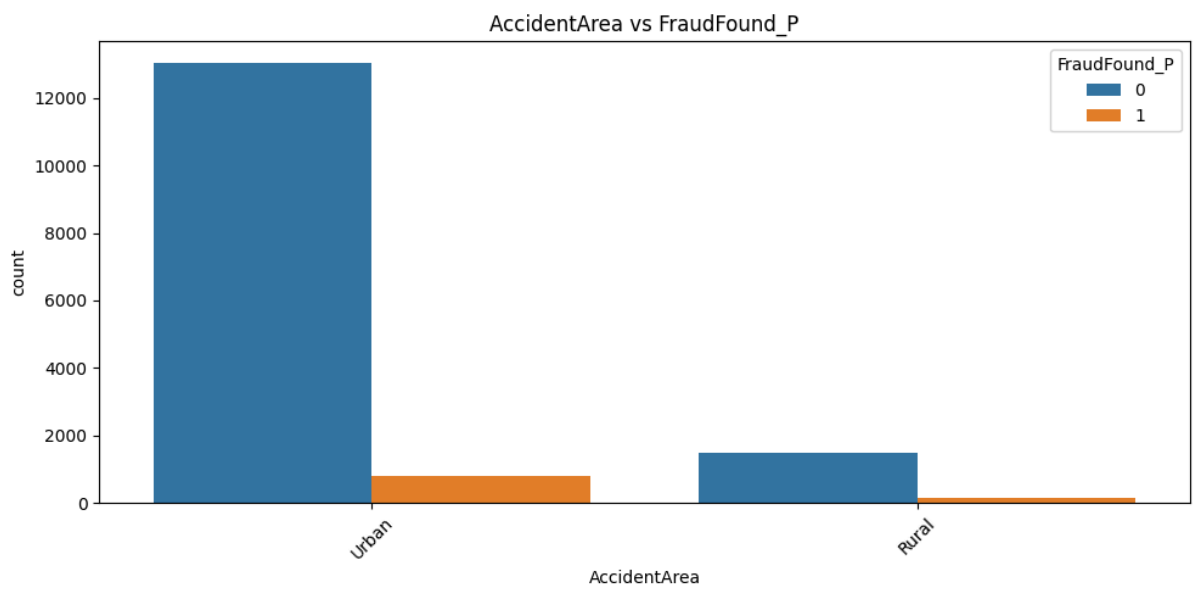
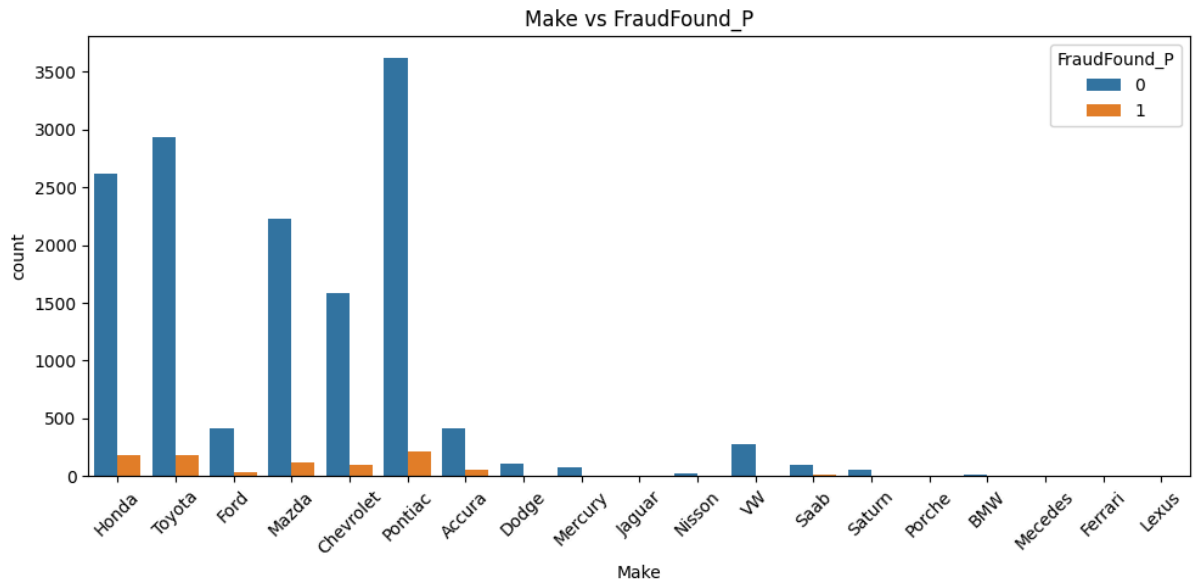
    if column != 'FraudFound_P':

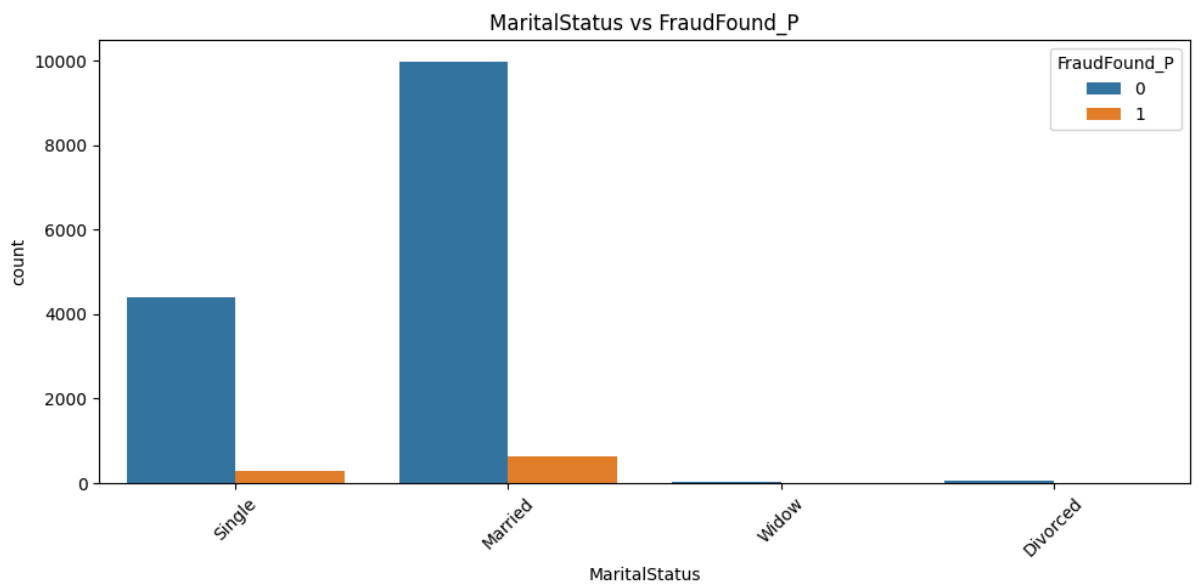
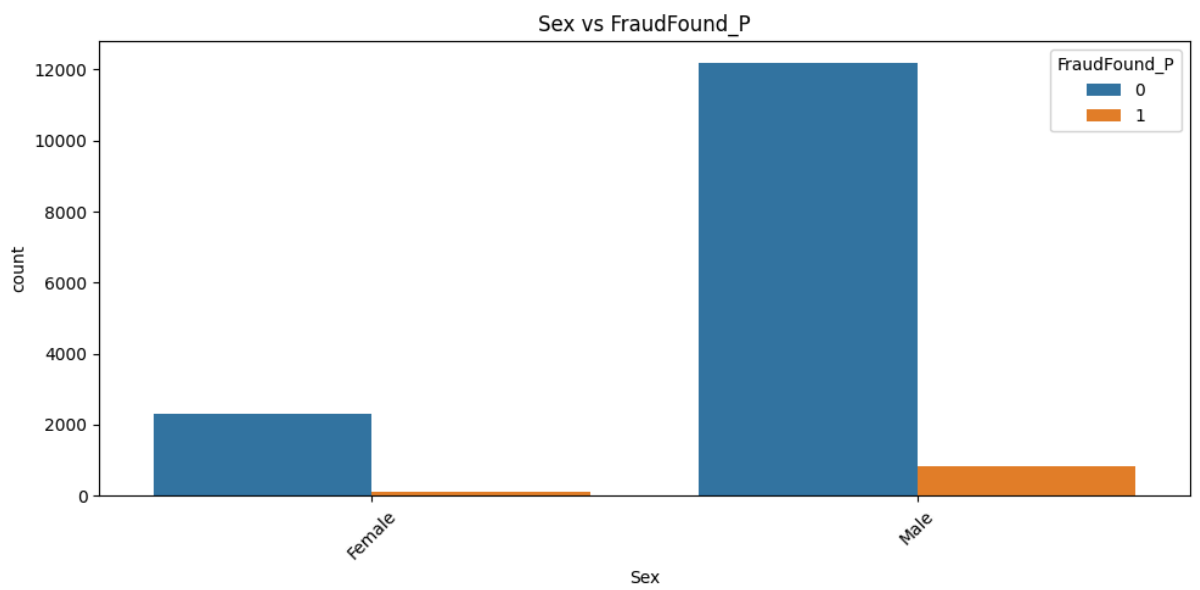
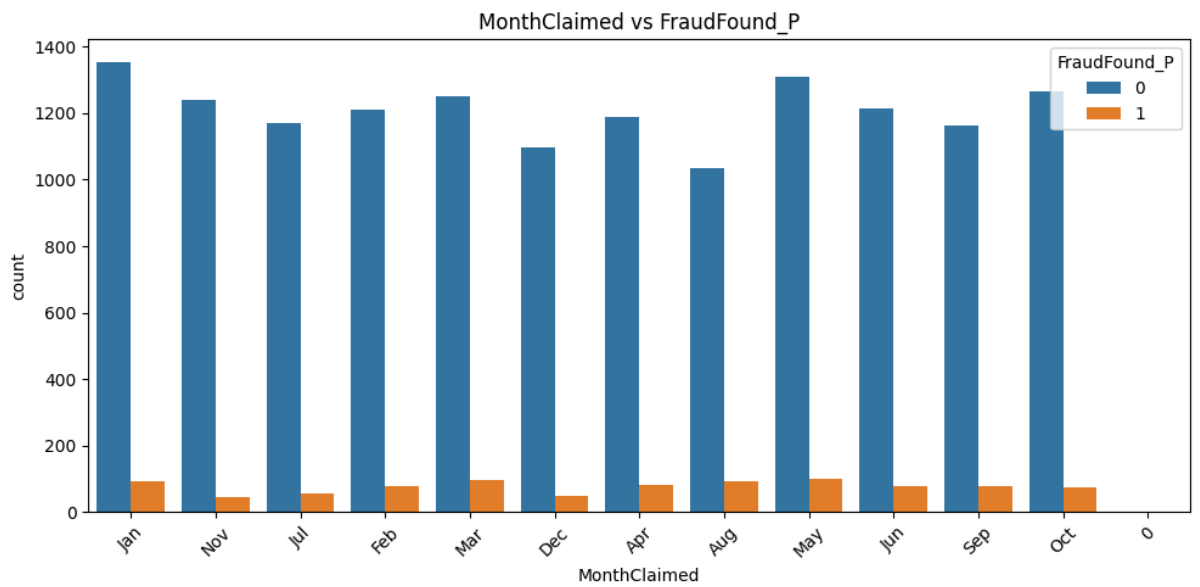
        plt.figure(figsize=(10, 5))

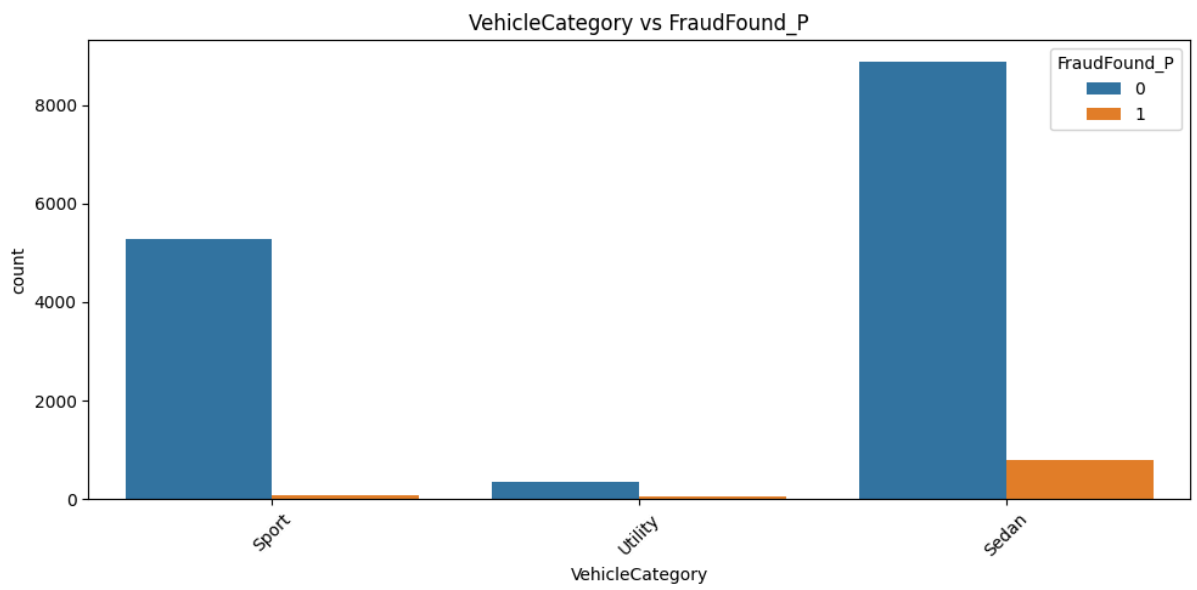
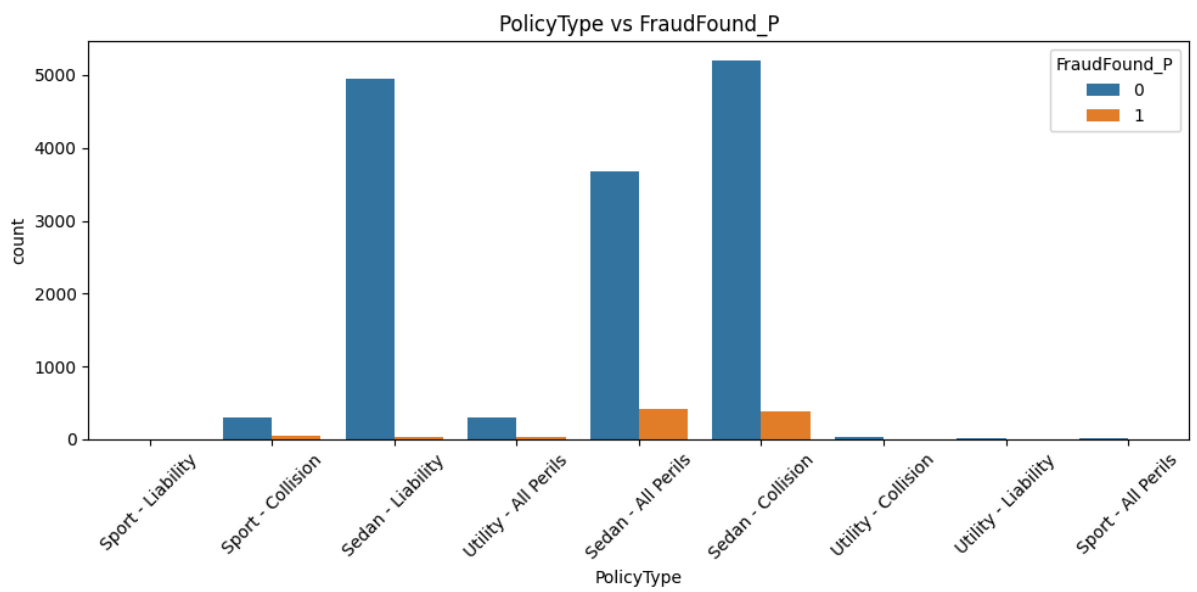
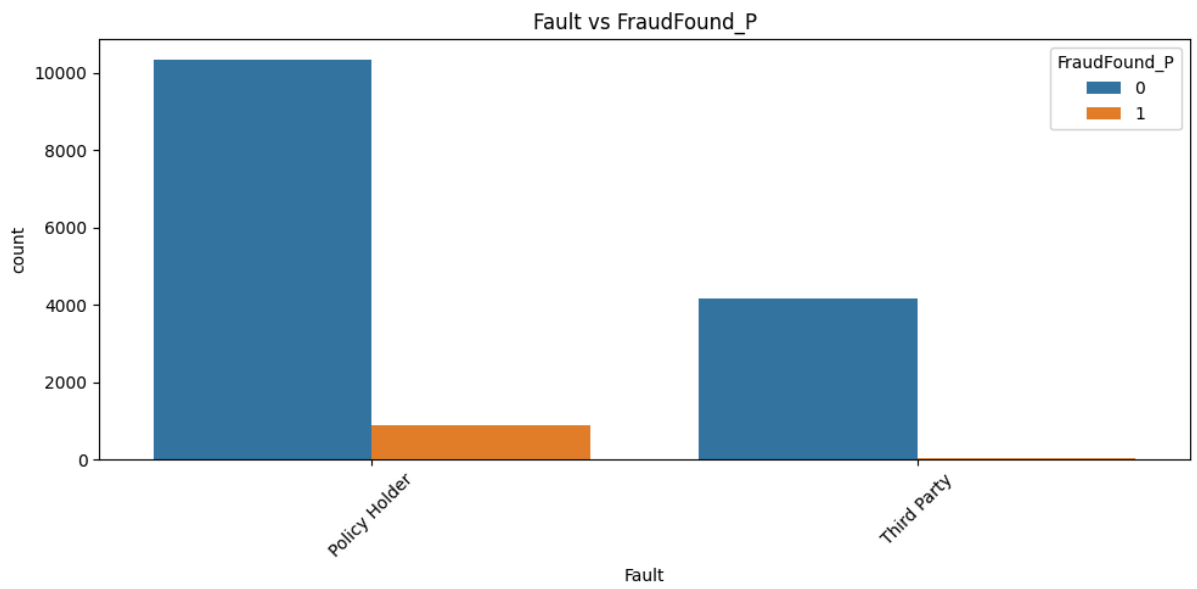
        sns.countplot(
            data=df,
            x=column,
            hue='FraudFound_P'
        )

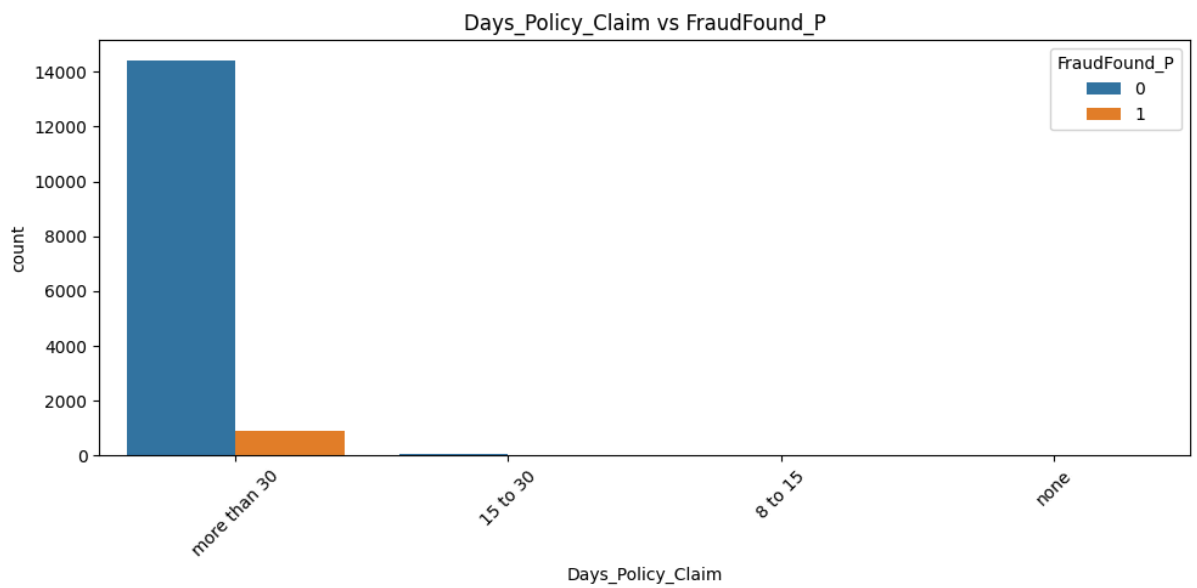
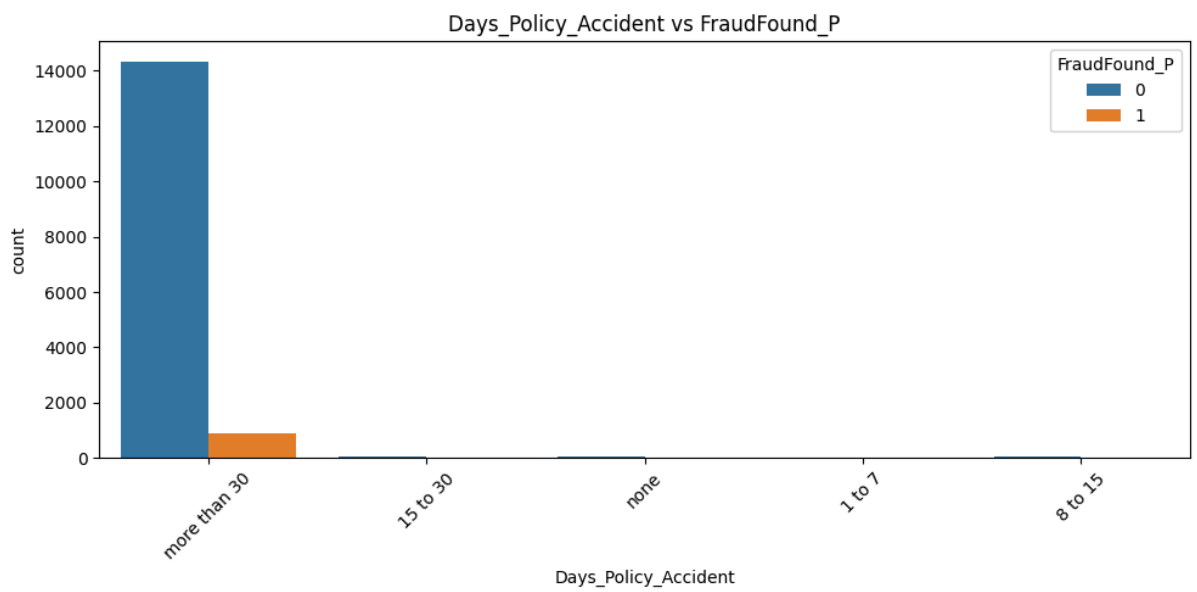
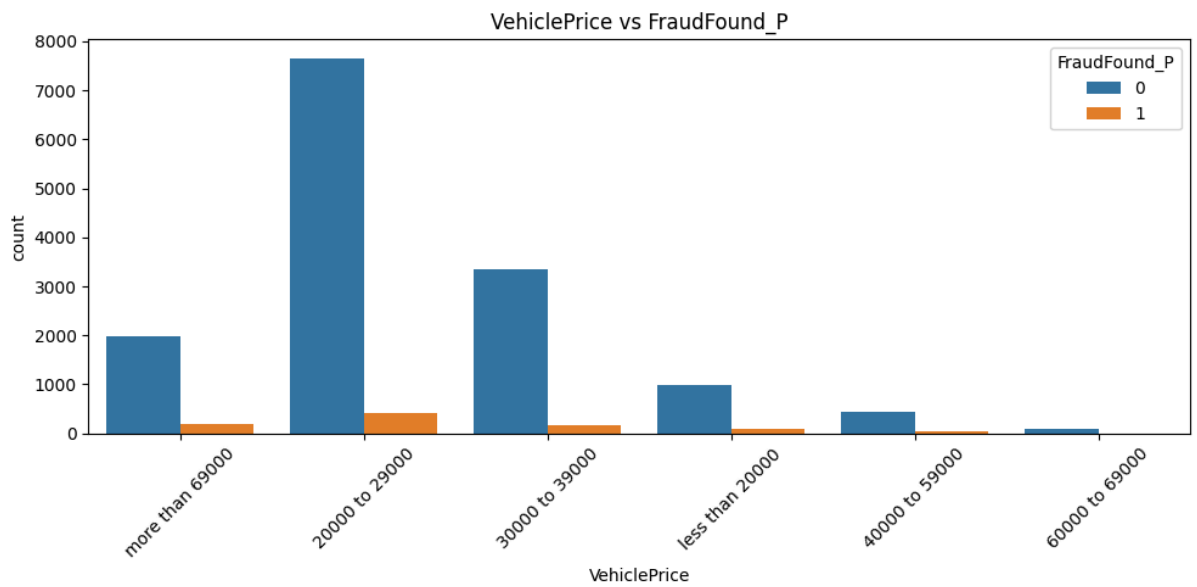
        plt.title(f'{column} vs FraudFound_P')
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()
```

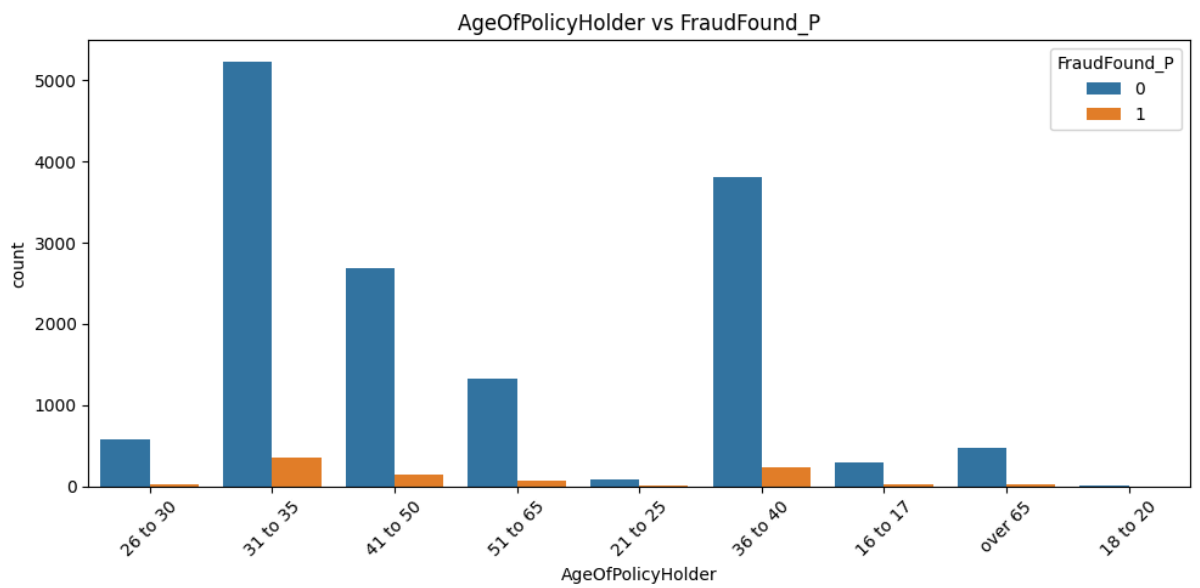
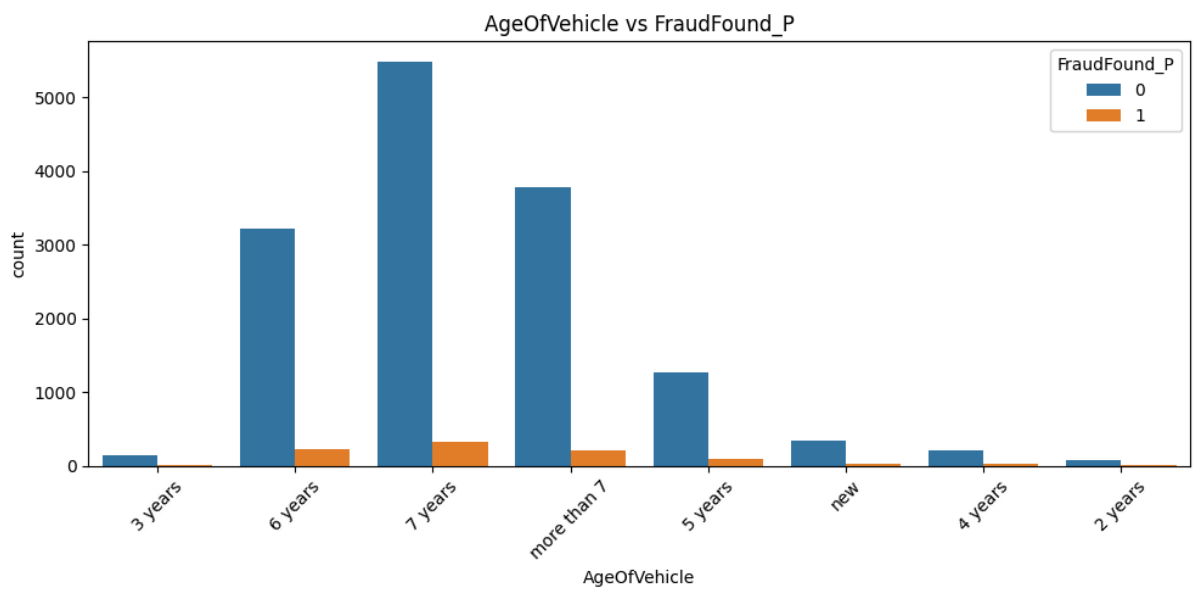
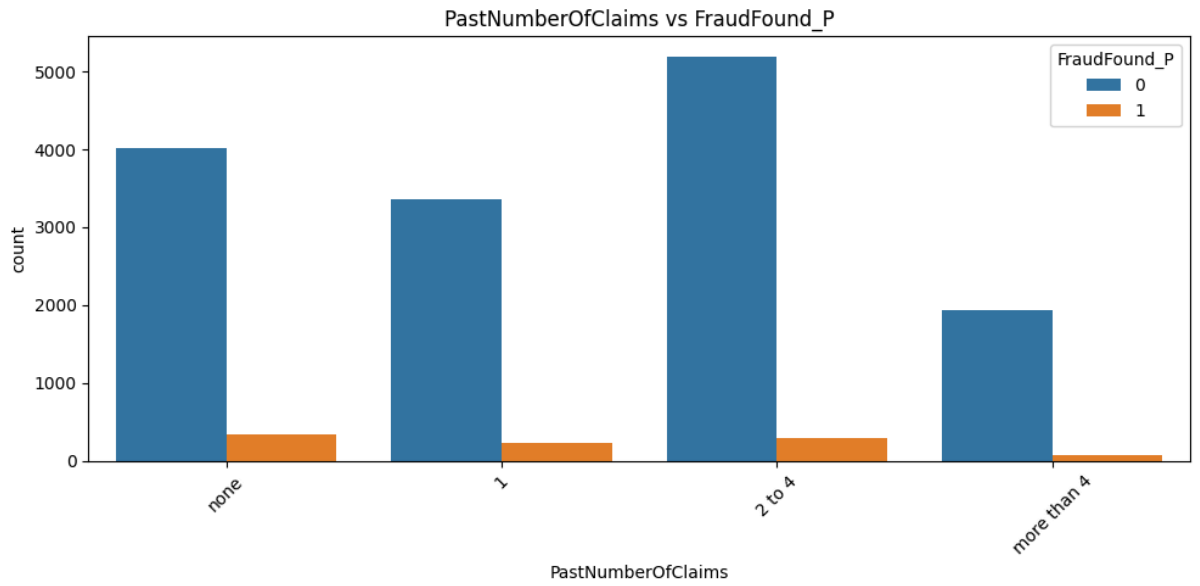


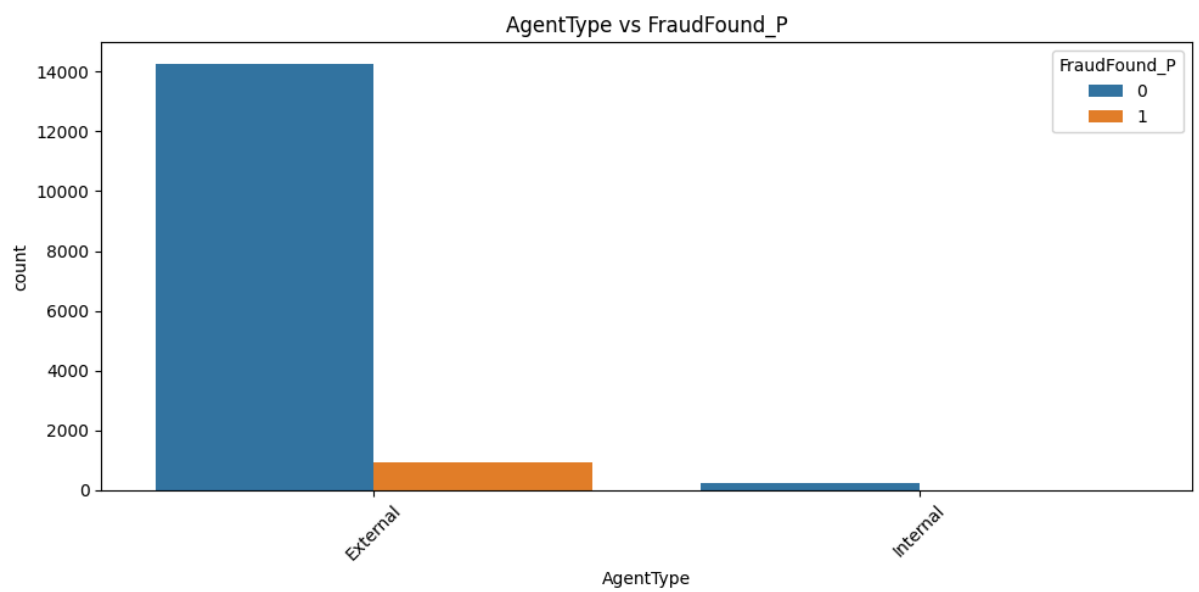
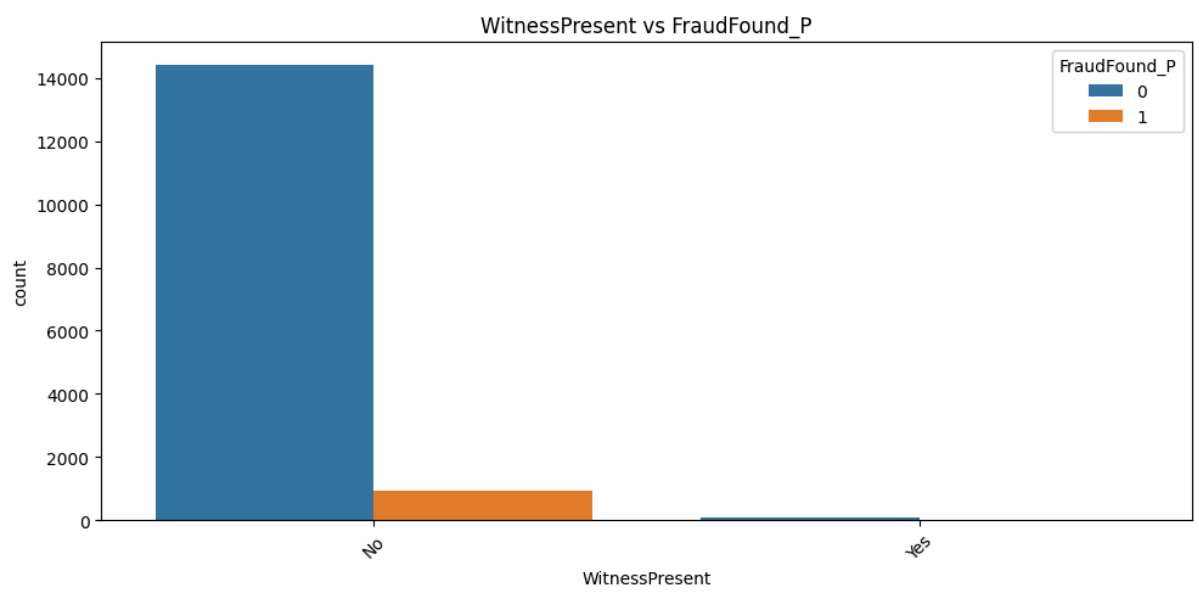
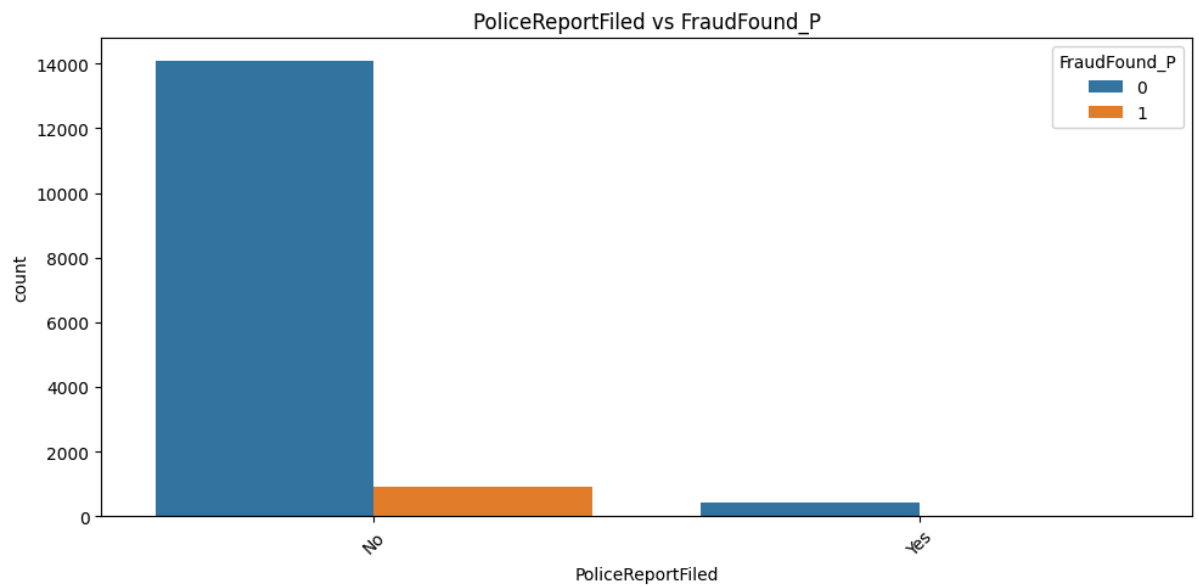


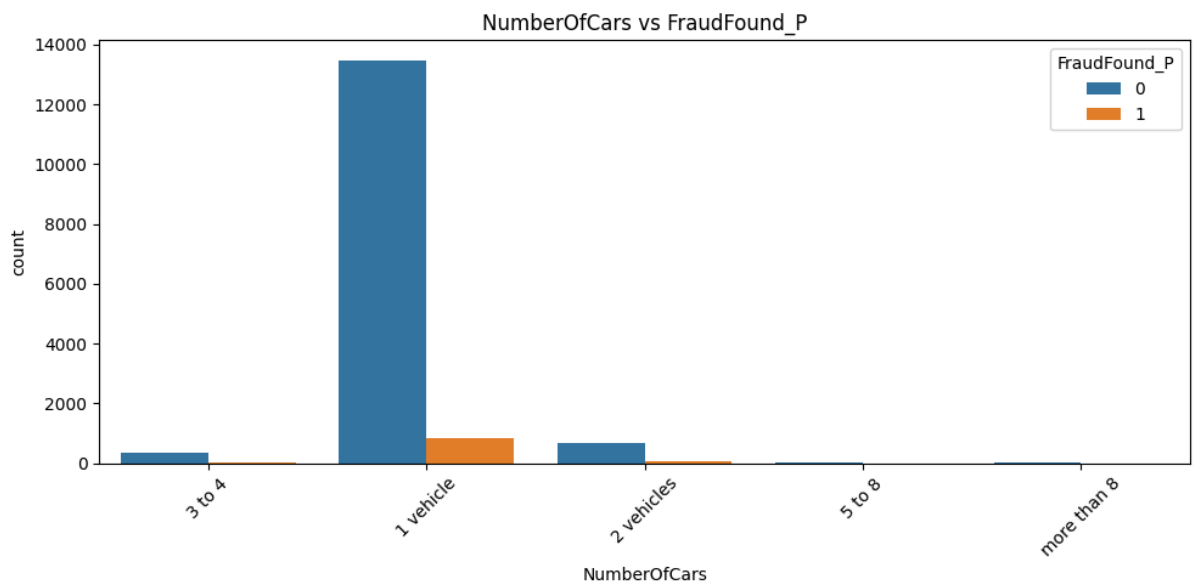
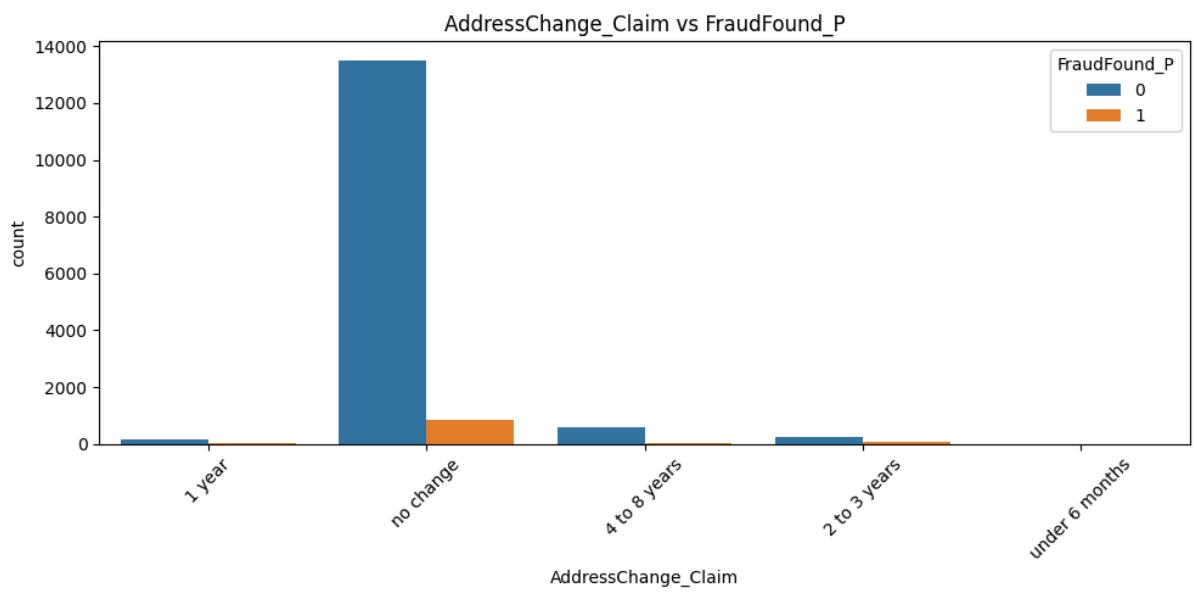
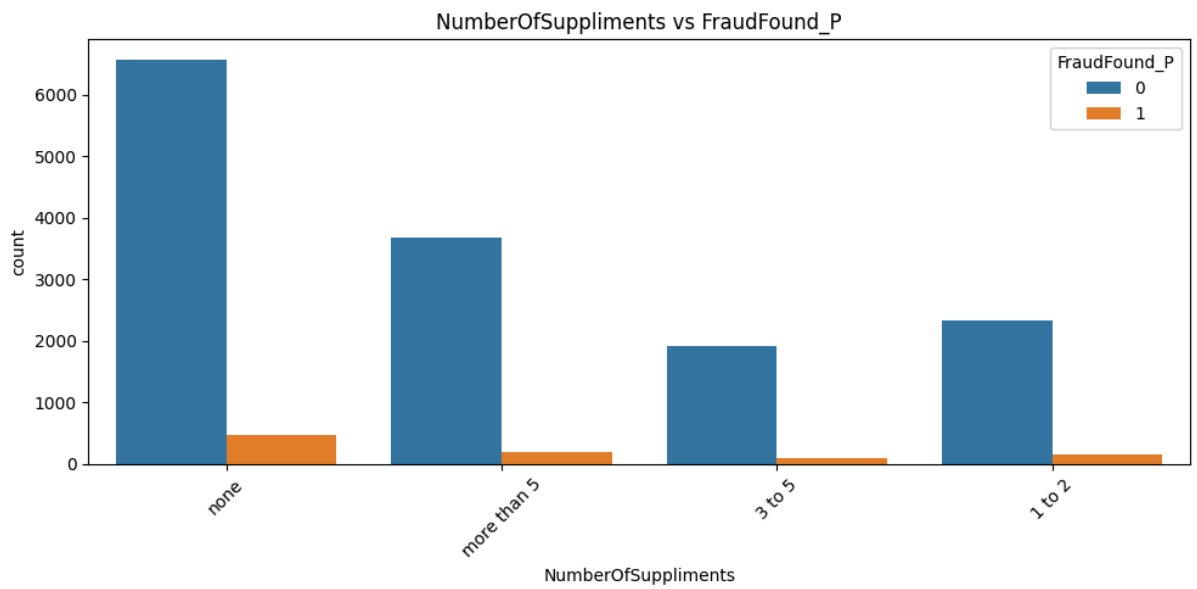


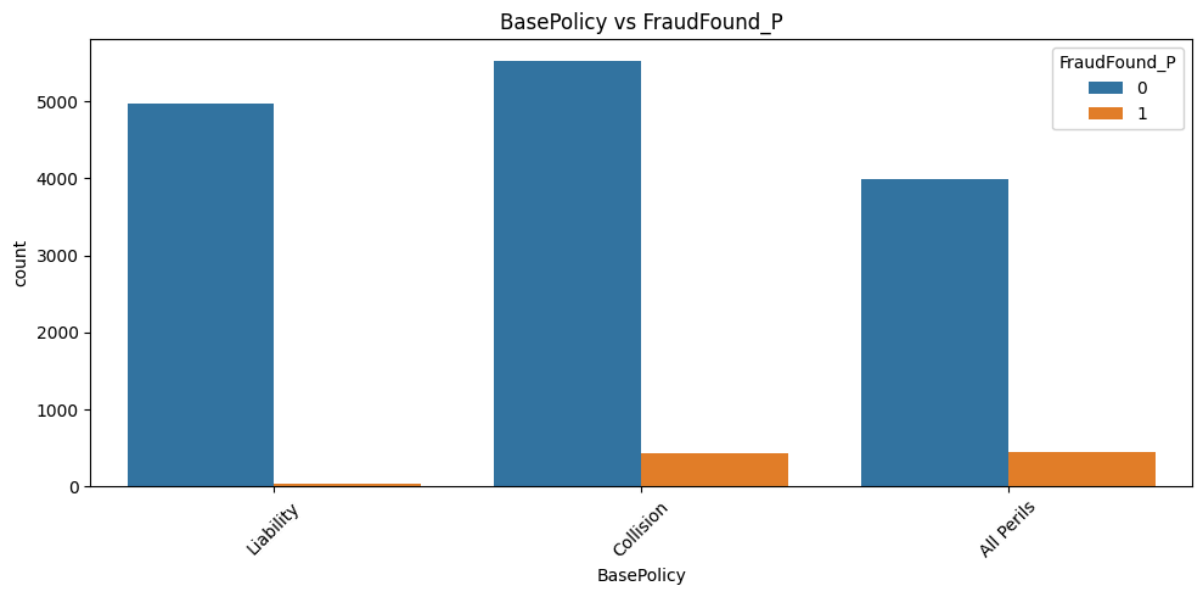












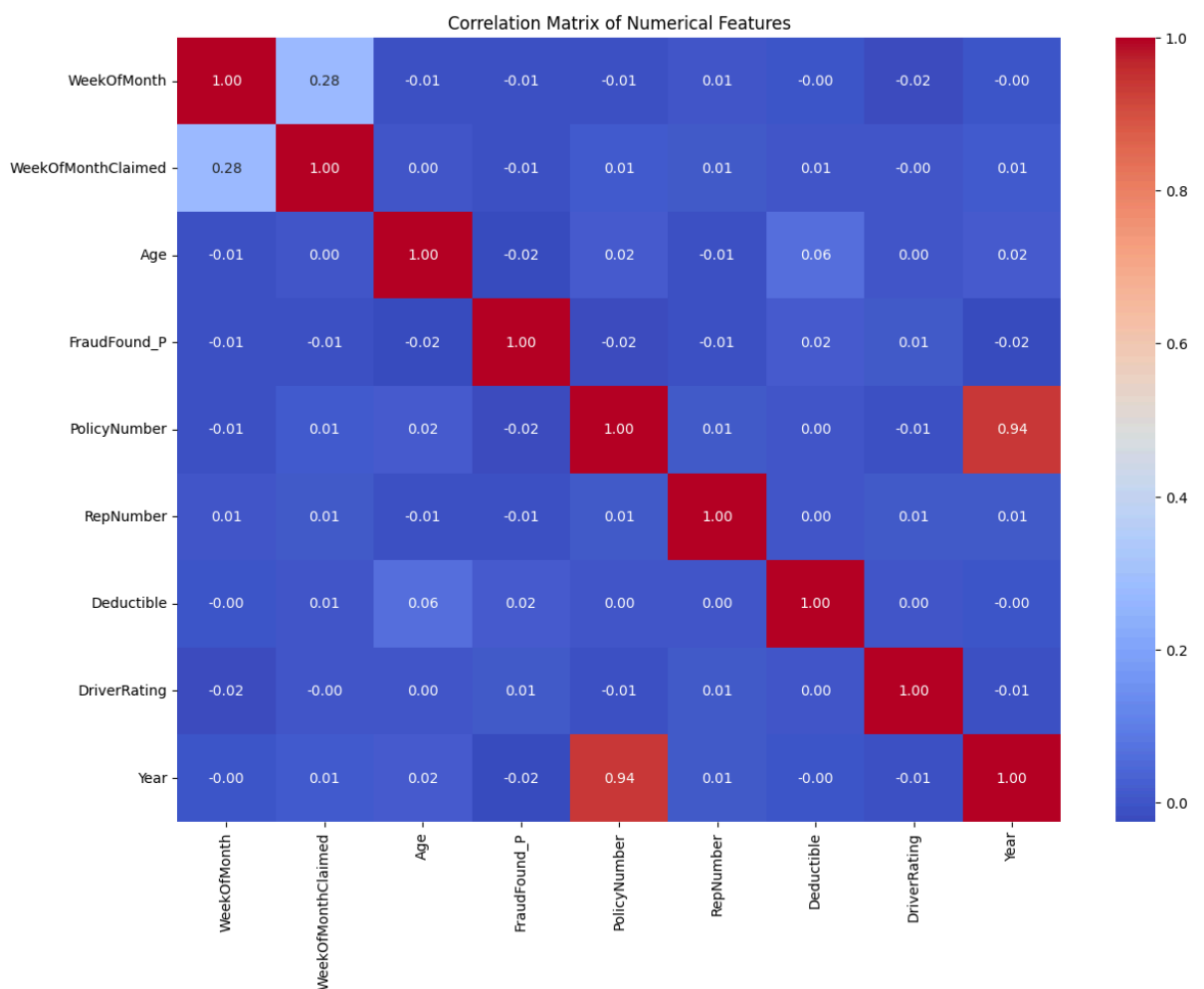
1. Correlation Matrix

```
In [34]: correlation_matrix = df1[numeric_columns].corr()

plt.figure(figsize=(14, 10))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



1. Encode categorical variables

```
In [39]: binary_cols = [col for col in df1.columns if df1[col].dtype == 'str' and
df1[col].nunique() == 2]
```

```
In [40]: print(binary_cols)
```

```
['AccidentArea', 'Sex', 'Fault', 'PoliceReportFiled', 'WitnessPresent',
'AgentType']
```

```
In [41]: binary_cols = [col for col in df1.columns if df1[col].dtype == 'str' and
df1[col].nunique() == 2]
for column in binary_cols:
    print(f"{column}: {df1[column].unique()}")
```

```
AccidentArea: <StringArray>
['Urban', 'Rural']
Length: 2, dtype: str
Sex: <StringArray>
['Female', 'Male']
Length: 2, dtype: str
Fault: <StringArray>
['Policy Holder', 'Third Party']
Length: 2, dtype: str
PoliceReportFiled: <StringArray>
['No', 'Yes']
Length: 2, dtype: str
WitnessPresent: <StringArray>
['No', 'Yes']
Length: 2, dtype: str
AgentType: <StringArray>
['External', 'Internal']
Length: 2, dtype: str
```

```
In [42]: from sklearn.preprocessing import LabelEncoder
# To encode/transform categorical column values to numerical values
le = LabelEncoder()

for col in binary_cols:
    # fit() - train label encoder with input samples
    # transform() - apply changes and transform existing dataframe samples.
    df1[col] = le.fit_transform(df1[col])
```

```
In [45]: non_binary_cols = [col for col in df1.columns if df1[col].dtype == 'str' and
df1[col].nunique() > 2]
```

```
In [46]: print(non_binary_cols)
```

```
['Month', 'DayOfWeek', 'Make', 'DayOfWeekClaimed', 'MonthClaimed',
'MaritalStatus', 'PolicyType', 'VehicleCategory', 'VehiclePrice',
'Days_Policy_Accident', 'Days_Policy_Claim', 'PastNumberOfClaims',
'AgeOfVehicle', 'AgeOfPolicyHolder', 'NumberOfSupplements',
'AddressChange_Claim', 'NumberOfCars', 'BasePolicy']
```

```
In [48]: df['AgeOfVehicle'].unique()
```

```
Out[48]: <StringArray>
[      '3 years',      '6 years',      '7 years', 'more than 7',      '5 years',
      'new',      '4 years',      '2 years']
Length: 8, dtype: str
```

```
In [50]: df1['AgeOfVehicle'] = df1['AgeOfVehicle'].str.replace(' years',  
    '').str.replace('more than', '').str.replace('new', '0')
```

```
In [51]: df1['AgeOfVehicle'].unique()
```

```
Out[51]: <StringArray>  
['3', '6', '7', ' 7', '5', '0', '4', '2']  
Length: 8, dtype: str
```

```
In [52]: df1['AgeOfVehicle'] = df1['AgeOfVehicle'].astype('float')
```

```
In [49]: df['AgeOfPolicyHolder'].unique()
```

```
Out[49]: <StringArray>  
['26 to 30', '31 to 35', '41 to 50', '51 to 65', '21 to 25', '36 to 40',  
 '16 to 17', 'over 65', '18 to 20']  
Length: 9, dtype: str
```

```
In [53]: for col in non_binary_cols:  
    # fit() - train label encoder with input samples  
    # transform() - apply changes and transform existing dataframe samples.  
    df1[col] = le.fit_transform(df1[col])
```

```
In [54]: df1.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 15420 entries, 0 to 15419
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Month                                15420 non-null  int64
1   WeekOfMonth                          15420 non-null  int64
2   DayOfWeek                            15420 non-null  int64
3   Make                                 15420 non-null  int64
4   AccidentArea                         15420 non-null  int64
5   DayOfWeekClaimed                     15420 non-null  int64
6   MonthClaimed                         15420 non-null  int64
7   WeekOfMonthClaimed                   15420 non-null  int64
8   Sex                                  15420 non-null  int64
9   MaritalStatus                       15420 non-null  int64
10  Age                                  15420 non-null  float64
11  Fault                                15420 non-null  int64
12  PolicyType                           15420 non-null  int64
13  VehicleCategory                      15420 non-null  int64
14  VehiclePrice                         15420 non-null  int64
15  FraudFound_P                         15420 non-null  int64
16  PolicyNumber                         15420 non-null  int64
17  RepNumber                            15420 non-null  int64
18  Deductible                           15420 non-null  int64
19  DriverRating                         15420 non-null  int64
20  Days_Policy_Accident                  15420 non-null  int64
21  Days_Policy_Claim                     15420 non-null  int64
22  PastNumberOfClaims                    15420 non-null  int64
23  AgeOfVehicle                          15420 non-null  int64
24  AgeOfPolicyHolder                     15420 non-null  int64
25  PoliceReportFiled                     15420 non-null  int64
26  WitnessPresent                        15420 non-null  int64
27  AgentType                             15420 non-null  int64
28  NumberOfSuppliments                   15420 non-null  int64
29  AddressChange_Claim                   15420 non-null  int64
30  NumberOfCars                          15420 non-null  int64
31  Year                                  15420 non-null  int64
32  BasePolicy                            15420 non-null  int64
dtypes: float64(1), int64(32)
memory usage: 3.9 MB
```

1. Split DataSet Into Train and Test Sets

```
In [69]: data = df1.copy()

print(data.shape)
```

```
(15420, 33)
```

```
In [70]: X = data.drop('FraudFound_P', axis=1)

Y = data['FraudFound_P']

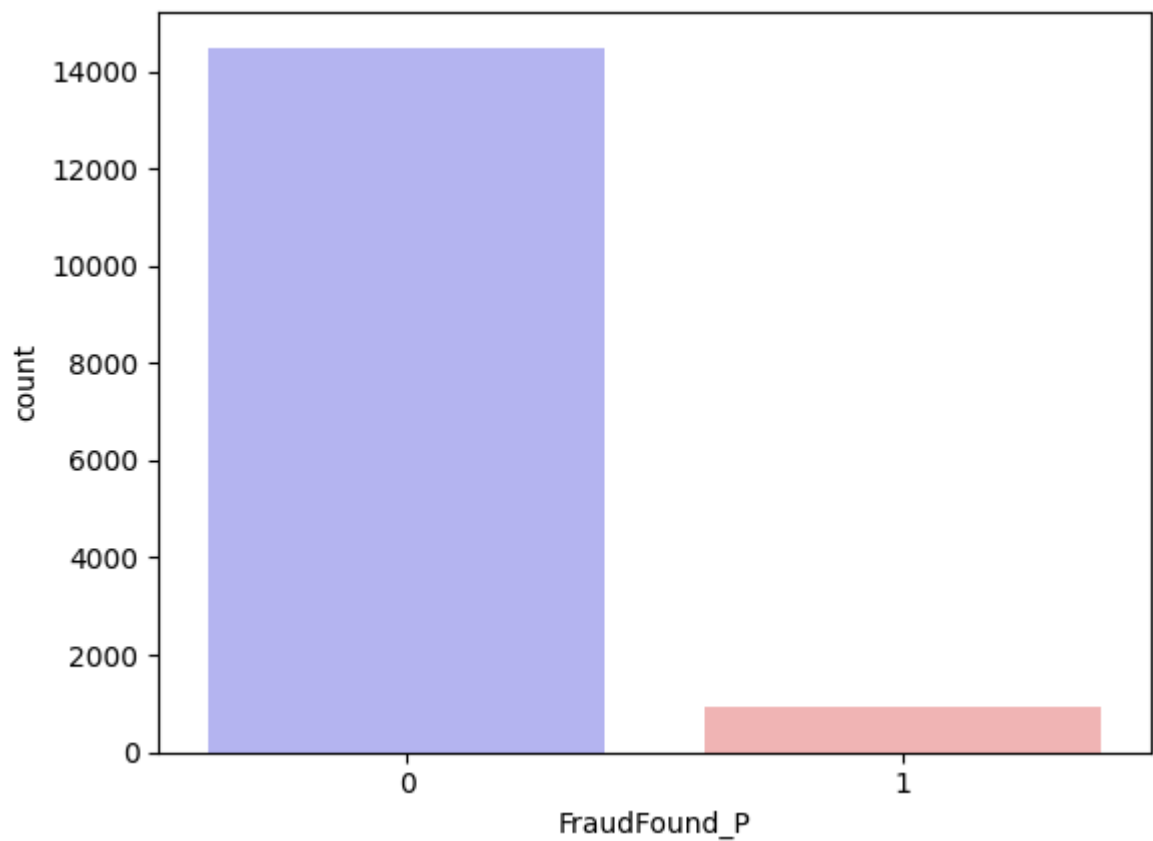
print("X shape:", X.shape)
print("Y shape:", Y.shape)
```

```
X shape: (15420, 32)
Y shape: (15420,)
```

1. Applying SMOTE

```
In [ ]: sns.countplot(x = 'FraudFound_P', data = df1, palette = 'bwr')
```

```
Out[57]: <Axes: xlabel='FraudFound_P', ylabel='count'>
```



```
In [62]: !pip install imbalanced-learn
```

```
Collecting imbalanced-learn
  Downloading imbalanced_learn-0.14.2-py3-none-any.whl.metadata (8.9 kB)
Requirement already satisfied: numpy<3, ≥1.25.2 in
c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages
(from imbalanced-learn) (2.4.6)
Requirement already satisfied: scipy<2, ≥1.11.4 in
c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages
(from imbalanced-learn) (1.15.2)
Requirement already satisfied: scikit-learn<2, ≥1.4.2 in
c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages
(from imbalanced-learn) (1.9.1)
Collecting sklearn-compat<0.2, ≥0.1.6 (from imbalanced-learn)
  Downloading sklearn_compat-0.1.6-py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: joblib<2, ≥1.2.0 in
c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages
(from imbalanced-learn) (1.6.0)
Requirement already satisfied: threadpoolctl<4, ≥2.0.0 in
c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages
(from imbalanced-learn) (3.7.0)
Requirement already satisfied: cloudpickle ≥3.0 in
c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages
(from joblib<2, ≥1.2.0→imbalanced-learn) (3.1.2)
Requirement already satisfied: narwhals ≥2.0.1 in
c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages
(from scikit-learn<2, ≥1.4.2→imbalanced-learn) (2.26.0)
Downloading imbalanced_learn-0.14.2-py3-none-any.whl (236 kB)
Downloading sklearn_compat-0.1.6-py3-none-any.whl (22 kB)
Installing collected packages: sklearn-compat, imbalanced-learn
Successfully installed imbalanced-learn-0.14.2 sklearn-compat-0.1.6
```

```
[notice] A new release of pip is available: 25.0.1 → 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [71]: from imblearn.over_sampling import SMOTE
smote = SMOTE()
transformed_feature, transformed_label = smote.fit_resample(X, Y)
```

```
In [72]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(transformed_feature,
transformed_label, test_size = 0.2,
                                                    random_state = 2)
```

1. Applying Logistic Regression

```
In [74]: # import logistic regression & model evaluation
from sklearn.linear_model import LogisticRegression
```

```
In [75]: # Hyperparameter - Performance tuning of ML Model
# solver : 'liblinear', 'lbfgs', 'newton-cg', 'newton-cholesky', 'sag',
#         'saga'
# max_iter : number of iterations
# C : inverse of regularization
# penalty : 'l1', 'l2' - avoids overfitting of regression model
logit_model = LogisticRegression(solver='lbfgs', max_iter = 10000, C = 1,
                                penalty = 'l2')
logit_model.fit(X_train, Y_train)
```

Out[75]: LogisticRegression(C=1, max_iter=10000, penalty='l2')

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



LogisticRegression

[?Documentation for LogisticRegression](#)iFitted

Parameters

	penalty_penalty: {'l1', 'l2', 'elasticnet', None}, default='l2' Specify the norm of the penalty: - `None`: no penalty is added; - `l2`: add an L2 penalty term and it is the default choice; - `l1`: add an L1 penalty term; - `elasticnet`: both L1 and L2 penalty terms are added. .. warning:: Some penalties may not work with some solvers. See the parameter `solver` below, to know the compatibility between the penalty and solver. .. versionadded:: 0.19 l1 penalty with SAGA solver (allowing 'multinomial' + L1) .. deprecated:: 1.8 `penalty` was deprecated in version 1.8 and will be removed in 1.10. Use `l1_ratio` and `C` instead. `l1_ratio=0` for `penalty='l2'`, `l1_ratio=1` for `penalty='l1'`, `l1_ratio` set to any float between 0 and 1 for `penalty='elasticnet'`, and `C=np.inf` for `penalty=None`.	'l2'
	max_iter max_iter: int, default=100 Maximum number of iterations taken for the solvers to converge.	10000
	C C: float, default=1.0 Inverse of regularization strength; must be a positive float. Like in support vector machines,	1

	smaller values specify stronger regularization. `C=np.inf` results in unpenalized logistic regression. For a visual example on the effect of tuning the `C` parameter with an L1 penalty, see: :ref:`sphx_glr_auto_examples_linear_model_plot_logistic_path.py`	
	l1_ratio l1_ratio: float, default=0.0 The Elastic-Net mixing parameter, with $0 \leq l1_ratio \leq 1$. Setting <code>l1_ratio=1</code> gives a pure L1-penalty, setting <code>l1_ratio=0</code> a pure L2-penalty. Any value between 0 and 1 gives an Elastic-Net penalty of the form <code>l1_ratio * L1 + (1 - l1_ratio) * L2</code>. .. warning:: Certain values of `l1_ratio`, i.e. some penalties, may not work with some solvers. See the parameter `solver` below, to know the compatibility between the penalty and solver. .. versionchanged:: 1.8 Default value changed from None to 0.0. .. deprecated:: 1.8 `None` is deprecated and will be removed in version 1.10. Always use <code>l1_ratio</code> to specify the penalty type.	0.0
	dual dual: bool, default=False Dual (constrained) or primal (regularized, see also :ref:`this equation <regularized-logistic-loss>`) formulation. Dual formulation is only implemented for L2 penalty with liblinear solver. Prefer <code>dual=False</code> when <code>n_samples > n_features</code>.	False
	tol tol: float, default=1e-4 Tolerance for stopping criteria.	0.0001
	fit_intercept fit_intercept: bool, default=True Specifies if a constant (a.k.a. bias or intercept) should be added to the decision function.	True

	intercept_scaling intercept_scaling: float, default=1 Useful only when the solver `liblinear` is used and `self.fit_intercept` is set to `True`. In this case, `x` becomes `[x, self.intercept_scaling]`. i.e. a "synthetic" feature with constant value equal to `intercept_scaling` is appended to the instance vector. The intercept becomes `intercept_scaling *` synthetic_feature_weight`. .. note:: The synthetic feature weight is subject to L1 or L2 regularization as all other features. To lessen the effect of regularization on synthetic feature weight (and therefore on the intercept). `intercept_scaling` has to be increased.	1
	class_weight class_weight: dict or 'balanced', default=None Weights associated with classes in the form `{class_label:</a>  <a href="#">weight}`. If not given, all classes are supposed to have weight one. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes *` np.bincount(y))`. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified. .. versionadded:: 0.17 *class_weight='balanced'*	None
	random_state random_state: int, RandomState instance, default=None Only used for `solver` == 'sag', 'saga' or 'liblinear' to shuffle the data. It has no effect on the other	None

	solvers. See :term:`Glossary` <random_state>` for details.	
	solver solver: {'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'}, default='lbfgs' Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects: - 'lbfgs' is a good default solver because it works reasonably well for a wide class of problems. - For :term:`multiclass` problems (<code>n_classes >= 3</code>), all solvers except 'liblinear' minimize the full multinomial loss, 'liblinear' will raise an error. - 'newton-cholesky' is a good choice for <code>n_samples > n_features * n_classes</code>, especially with one-hot encoded categorical features with rare categories. Be aware that the memory usage of this solver has a quadratic dependency on <code>n_features * n_classes</code> because it explicitly computes the full Hessian matrix. - For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones; - 'liblinear' can only handle binary classification by default. To apply a one-versus-rest scheme for the multiclass setting one can wrap it with the :class:`~sklearn.multiclass.OneVsRestClassifier`. .. warning:: The choice of the algorithm depends on the penalty chosen (<code>l1_ratio=0` for L2-penalty, 'l1_ratio=1` for L1-penalty and `0 < l1_ratio < 1` for Elastic-Net) and on (multinomial) multiclass support:</code>	'lbfgs'

	===== ===== ===== solver l1_ratio multinomial multiclass ===== ===== ===== 'lbfgs' l1_ratio=0 yes 'liblinear' l1_ratio=1 or l1_ratio=0 no 'newton-cg' l1_ratio=0 yes 'newton-cholesky' l1_ratio=0 yes 'sag' l1_ratio=0 yes 'saga' 0<=l1_ratio<=1 yes ===== ===== ===== .. note:: 'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from :mod:`sklearn.preprocessing`. .. seealso:: Refer to the :ref:`User Guide <Logistic_regression>` for more information regarding :class:`LogisticRegression` and more specifically the :ref:`Table <logistic_regression_solvers>` summarizing solver/penalty supports. .. versionadded:: 0.17 Stochastic Average Gradient (SAG) descent solver. Multinomial support in version 0.18. .. versionadded:: 0.19 SAGA solver. .. versionchanged:: 0.22 The default solver changed from 'liblinear' to 'lbfgs' in 0.22. .. versionadded:: 1.2 newton-cholesky solver. Multinomial support in version 1.6.	
	verbose verbose: int, default=0 For the liblinear and lbfgs solvers set verbose to any positive number for verbosity.	0
	warm_start warm_start: bool, default=False	False

	When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution. Useless for liblinear solver. See :term:`the Glossary <warm_start>`. .. versionadded:: 0.17 *warm_start* to support *lbfgs*, *newton-cg*, *sag*, *saga* solvers.	
	n_jobs n_jobs: int, default=None Does not have any effect. .. deprecated:: 1.8 `n_jobs` is deprecated in version 1.8 and will be removed in 1.10.	None

Fitted attributes

Name	Type	Value
classes_ classes_ : ndarray of shape (n_classes,) A list of class labels known to the classifier.	ndarray[int64](2,)	[0,1]
coef_ coef_ : ndarray or CSR matrix of shape (1, n_features) or (n_classes, n_features) Coefficients of the features in the decision function. `coef_` is of shape (1, n_features) when the given problem is binary. By default, it will be created as a dense array, but can be turned to sparse (CSR format) through :meth:`sparsify` (which can be beneficial under L1 regularization when many coefficients are zero), and back to dense through :meth:`densify`.	ndarray[float64](1, 32)	[[0.02,-0.06,-0.11,...,-0.67, 0.01,-2.52]]
feature_names_in_ feature_names_in_ : ndarray of shape (n_features_in_,) Names of features seen during :term:`fit`. Defined only when `X` has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](32,)	['Month','WeekOfMonth','DayOfWeek',..., 'NumberOfCars','Year','BasePolicy']

intercept_ : ndarray of shape (1,) or (n_classes,) Intercept (a.k.a. bias) added to the decision function. If 'fit_intercept' is set to False, the intercept is set to zero. 'intercept_' is of shape (1,) when the given problem is binary.	ndarray[float64](1,)	[0.]
n_features_in_ : int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	32
n_iter_ : ndarray of shape (1,) Actual number of iterations for all classes. .. versionchanged:: 0.20 In SciPy <= 1.0.0 the number of lbfgs iterations may exceed ``max_iter``. ``n_iter`` will now report at most ``max_iter``.	ndarray[int32](1,)	[10000]

```
In [94]: logit_model.score(X_test, Y_test)
```

```
Out[94]: 0.8649767201241594
```

1. Applying Decision Tree

```
In [91]: from sklearn.tree import DecisionTreeClassifier
# hyperparameter tuning is setup the users to improve performance of ml
# model s
# criterion = 'entropy' measures information gain or 'gini' measures
# impurity. This is by default
# max_depth = depth of decision tree
# ccp_alpha = Post pruning - First allow the tree to grow and then cut off
# branches
# First grow decision tree then remove unnecessary branches that do not
# improve model performance
dt_clf = DecisionTreeClassifier()
dt_clf.fit(X_train, Y_train)
```

Out[91]: DecisionTreeClassifier()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



DecisionTreeClassifier

[?Documentation for DecisionTreeClassifier](#)Fitted

Parameters

	criterion criterion: {"gini", "entropy", "log_loss"}, default="gini" The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:`tree_mathematical_formulation`.	'gini'
	splitter splitter: {"best", "random"}, default="best" The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split among considered features for this split.	'best'
	max_depth max_depth: int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.	None
	min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider <code>min_samples_split</code> as the minimum number. - If float, then <code>min_samples_split</code> is a fraction and <code>ceil(min_samples_split * </code>	2

	<u><code>n_samples</code></u>)` are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	
	<u><code>min_samples_leaf</code></u> <u><code>min_samples_leaf</code>: int or float, default=1</u> The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least ``<code>min_samples_leaf</code>`` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression. - If int, then consider ``<code>min_samples_leaf</code>`` as the minimum number. - If float, then ``<code>min_samples_leaf</code>`` is a fraction and ``<code>ceil(min_samples_leaf * n_samples)</code>`` are the minimum number of samples for each node. .. versionchanged:: 0.18 Added float values for fractions.	1
	<u><code>min_weight_fraction_leaf</code></u> <u><code>min_weight_fraction_leaf</code>: float, default=0.0</u> The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when <code>sample_weight</code> is not provided.	0.0
	<u><code>max_features</code></u> <u><code>max_features</code>: int, float or {"sqrt", "log2"}, default=None</u> The number of features to consider when looking for the best split: - If int, then consider ``<code>max_features</code>`` features at each split. - If float, then ``<code>max_features</code>`` is a fraction and ``<code>max(1, int(max_features * n_features_in_))</code>`` features are considered at each split. - If "sqrt", then	None

	`max_features=sqrt(n_features)`. - If "log2", then `max_features=log2(n_features)`. - If None, then `max_features=n_features`. Note: splitting may inspect more than ``max_features`` features if needed to find a valid split.	
	random_state random_state: int, RandomState instance or None, default=None Controls the randomness of the estimator. The features are always randomly permuted at each split, even if ``splitter`` is set to ``"best"``. When ``max_features < n_features``, the algorithm will select ``max_features`` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if ``max_features=n_features``. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, ``random_state`` has to be fixed to an integer. See :term:`Glossary` <random_state>` for details.	None
	max_leaf_nodes max_leaf_nodes: int, default=None Grow a tree with ``max_leaf_nodes`` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	None
	min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value.	0.0

	The weighted impurity decrease equation is the following:: $\frac{N_t}{N} * (\text{impurity} - \frac{N_{t_R}}{N_t} * \text{right_impurity} - \frac{N_{t_L}}{N_t} * \text{left_impurity})$ where ``N`` is the total number of samples, ``N_t`` is the number of samples at the current node, ``N_{t_L}`` is the number of samples in the left child, and ``N_{t_R}`` is the number of samples in the right child. ``N``, ``N_t``, ``N_{t_R}`` and ``N_{t_L}`` all refer to the weighted sum, if ``sample_weight`` is passed. .. versionadded:: 0.19	
	class_weight class_weight: dict, list of dict or "balanced", default=None Weights associated with classes in the form ``{class_label: weight}``. If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as ``n_samples / (n_classes * np.bincount(y))`` For multi-output, the weights of each column of y will be multiplied. Note that these weights will be	None

	multiplied with sample_weight (passed through the fit method) if sample_weight is specified.	
	ccp_alpha ccp_alpha : non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ``ccp_alpha`` will be chosen. By default, no pruning is performed. See :ref:`minimal_cost_complexity_pruning` for details. See :ref:`sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py` for an example of such pruning. .. versionadded:: 0.22	0.0
	monotonic_cst monotonic_cst : array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when `n_classes > 2`), - multioutput classifications (i.e. when `n_outputs_ > 1`). The constraints hold over the probability of the positive class. Read more in the :ref:`User Guide <monotonic_cst_gbdt>`. .. versionadded:: 1.4	None

Fitted attributes

Name	Type	Value
classes_ classes_ : ndarray of shape (n_classes,) or list of ndarray	<code>ndarray[int64](2,)</code>	[0,1]

The classes labels (single output problem). or a list of arrays of class labels (multi-output problem).		
feature_importances_ feature_importances_ : ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance [4]. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:'sklearn.inspection.permutation_importance' as an alternative.	ndarray[float64](32,)	[0.02,0.01,0.02,...,0.,0.,0.26]
feature_names_in_ feature_names_in_ : ndarray of shape (n_features_in_,) Names of features seen during :term:'fit'. Defined only when 'X' has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](32,)	['Month','WeekOfMonth','DayOfWeek',...,'NumberOfCars','Year','BasePolicy']
max_features_ max_features_ : int The inferred value of max_features.	int	32
n_classes_ n_classes_ : int or list of int The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).	int64	np.int64(2)
n_features_in_ n_features_in_ : int Number of features seen during :term:'fit'. .. versionadded:: 0.24	int	32

n_outputs_ n_outputs_ : int The number of outputs when ``fit`` is performed.	int	1
tree_ tree_ : Tree instance The underlying Tree object. Please refer to ``help(sklearn.tree._tree.Tree)`` for attributes of Tree object and :ref:`sphx_glr_auto_examples_tree_plot_unveil_tree_structure.py` for basic usage of these attributes.	Tree	<sklearn.tree...001D58F4CB4B0>

```
In [93]: # training score
dt_clf.score(X_train, Y_train)
```

```
Out[93]: 1.0
```

```
In [96]: dt_clf.score(X_test, Y_test)
```

```
Out[96]: 0.9161924469736161
```

```
In [97]: dt_clf.get_depth()
```

```
Out[97]: 32
```

```
In [98]: path = dt_clf.cost_complexity_pruning_path(X_train, Y_train)
ccp_alphas = path.ccp_alphas
impurities = path.impurities
```

In [99]: `ccp_alphas`

[illegible]

6.30507636e-05, 6.33440734e-05, 6.42789197e-05, 6.46691097e-05,
6.46691097e-05, 6.46691097e-05, 6.46691097e-05, 6.46691097e-05,
6.46691097e-05, 6.46691097e-05, 6.46691097e-05, 6.46691097e-05,
6.46691097e-05, 6.46691097e-05, 6.46691097e-05, 6.46691097e-05,
6.46691097e-05, 6.46691097e-05, 6.46691097e-05, 6.46691097e-05,
6.46691097e-05, 6.46691097e-05, 6.46691097e-05, 6.54464471e-05,
6.56956035e-05, 6.56956035e-05, 6.56956035e-05, 6.59510442e-05,
6.63681340e-05, 6.68800195e-05, 6.68800195e-05, 6.68800195e-05,
6.70642619e-05, 6.71651104e-05, 6.78528645e-05, 6.78749288e-05,
6.79352264e-05, 6.86778382e-05, 6.87204089e-05, 6.89803837e-05,
6.89803837e-05, 6.89803837e-05, 6.89803837e-05, 6.89803837e-05,
6.89803837e-05, 6.89803837e-05, 6.89803837e-05, 6.89803837e-05,
6.89803837e-05, 6.89803837e-05, 6.89803837e-05, 6.89803837e-05,
6.89803837e-05, 6.89803837e-05, 6.89803837e-05, 6.89803837e-05,
6.89803837e-05, 6.93909812e-05, 6.99652435e-05, 7.05481197e-05,
7.05481197e-05, 7.07491115e-05, 7.08965055e-05, 7.11360207e-05,
7.13262240e-05, 7.14042074e-05, 7.18390302e-05, 7.18545664e-05,
7.18545664e-05, 7.18545664e-05, 7.18545664e-05, 7.18545664e-05,
7.18545664e-05, 7.18545664e-05, 7.18545664e-05, 7.18545664e-05,
7.18545664e-05, 7.18545664e-05, 7.18545664e-05, 7.18545664e-05,
7.18545664e-05, 7.24294029e-05, 7.24294029e-05, 7.27178691e-05,
7.28630515e-05, 7.29600212e-05, 7.36270699e-05, 7.36813774e-05,
7.39075540e-05, 7.39075540e-05, 7.39075540e-05, 7.39075540e-05,
7.39075540e-05, 7.39075540e-05, 7.39075540e-05, 7.39075540e-05,
7.39075540e-05, 7.39075540e-05, 7.39075540e-05, 7.39075540e-05,
7.40102033e-05, 7.41128527e-05, 7.47810069e-05, 7.47876791e-05,
7.52323451e-05, 7.54472947e-05, 7.54472947e-05, 7.54472947e-05,
7.54472947e-05, 7.57466887e-05, 7.66448708e-05, 7.66448708e-05,
7.66448708e-05, 7.66448708e-05, 7.66448708e-05, 7.66448708e-05,
7.68502574e-05, 7.69870354e-05, 7.72014036e-05, 7.72121437e-05,
7.72612456e-05, 7.74117914e-05, 7.74117914e-05, 7.75558139e-05,
7.75589391e-05, 7.76029317e-05, 7.76029317e-05, 7.76029317e-05,
7.76029317e-05, 7.76029317e-05, 7.76029317e-05, 7.77823167e-05,
7.79737939e-05, 7.79737939e-05, 7.79737939e-05, 7.81522446e-05,
7.82161463e-05, 7.83867997e-05, 7.83867997e-05, 7.83867997e-05,
7.83867997e-05, 7.83867997e-05, 7.83867997e-05, 7.85267761e-05,
7.90400230e-05, 7.90400230e-05, 7.90400230e-05, 7.90400230e-05,
7.90400230e-05, 7.94795317e-05, 7.95927504e-05, 7.95927504e-05,
7.97893086e-05, 7.98138414e-05, 7.98281511e-05, 7.98720232e-05,
7.99496558e-05, 8.00665168e-05, 8.01043692e-05, 8.03339558e-05,
8.03920434e-05, 8.04771143e-05, 8.04771143e-05, 8.04771143e-05,
8.04771143e-05, 8.04771143e-05, 8.04771143e-05, 8.05206625e-05,
8.08235008e-05, 8.08363872e-05, 8.08363872e-05, 8.08363872e-05,
8.11366181e-05, 8.11872065e-05, 8.12834461e-05, 8.13853068e-05,
8.14351752e-05, 8.14351752e-05, 8.15444379e-05, 8.18053165e-05,
8.18704681e-05, 8.19142056e-05, 8.20193587e-05, 8.20532790e-05,
8.21195044e-05, 8.21195044e-05, 8.21886417e-05, 8.22050456e-05,
8.23841365e-05, 8.26327513e-05, 8.26327513e-05, 8.27764604e-05,
8.28038336e-05, 8.28257321e-05, 8.29565555e-05, 8.30319433e-05,
8.31459982e-05, 8.32521872e-05, 8.33512970e-05, 8.34280020e-05,

8.34657401e-05, 8.35309334e-05, 8.36125863e-05, 8.36125863e-05,
8.37294789e-05, 8.38303274e-05, 8.41224192e-05, 8.48568212e-05,
8.48592479e-05, 8.49656190e-05, 8.52782773e-05, 8.55381751e-05,
8.56467851e-05, 8.56635235e-05, 8.59060615e-05, 8.62254796e-05,
8.66104148e-05, 8.84363894e-05, 8.84363894e-05, 8.84363894e-05,
8.88603435e-05, 8.88689587e-05, 8.88689587e-05, 9.00748314e-05,
9.03569873e-05, 9.05367536e-05, 9.07484333e-05, 9.08832460e-05,
9.24351335e-05, 9.42668510e-05, 9.43254489e-05, 9.50635913e-05,
9.55241224e-05, 9.55978144e-05, 9.70108741e-05, 9.75011193e-05,
9.76820119e-05, 9.76820119e-05, 1.00309605e-04, 1.01024099e-04,
1.01362488e-04, 1.02709762e-04, 1.03470576e-04, 1.03470576e-04,
1.03470576e-04, 1.03470576e-04, 1.03470576e-04, 1.03470576e-04,
1.04577945e-04, 1.04594316e-04, 1.05461842e-04, 1.06017001e-04,
1.06645333e-04, 1.06660336e-04, 1.07150271e-04, 1.07535491e-04,
1.08801011e-04, 1.08916395e-04, 1.09380528e-04, 1.09492673e-04,
1.09741520e-04, 1.10500157e-04, 1.12667960e-04, 1.12756396e-04,
1.12784129e-04, 1.13475838e-04, 1.14240347e-04, 1.14414579e-04,
1.14460972e-04, 1.14517671e-04, 1.14967306e-04, 1.14967306e-04,
1.14967306e-04, 1.15179977e-04, 1.15337742e-04, 1.15734983e-04,
1.15958404e-04, 1.16404397e-04, 1.16404397e-04, 1.16846579e-04,
1.16966738e-04, 1.17212761e-04, 1.17247420e-04, 1.17362458e-04,
1.18662684e-04, 1.19228915e-04, 1.19757611e-04, 1.19890823e-04,
1.20128377e-04, 1.20235597e-04, 1.20595076e-04, 1.20595076e-04,
1.20759850e-04, 1.20831765e-04, 1.21594603e-04, 1.22631793e-04,
1.23179257e-04, 1.23179257e-04, 1.23179257e-04, 1.23282249e-04,
1.23459209e-04, 1.23544360e-04, 1.24964463e-04, 1.25311205e-04,
1.26412015e-04, 1.26454561e-04, 1.27342259e-04, 1.27926856e-04,
1.28311726e-04, 1.28311726e-04, 1.29338219e-04, 1.29338219e-04,
1.29338219e-04, 1.29338219e-04, 1.29788875e-04, 1.30184221e-04,
1.30644666e-04, 1.30740559e-04, 1.30775311e-04, 1.31174665e-04,
1.31391207e-04, 1.31689823e-04, 1.32757307e-04, 1.33076436e-04,
1.33120039e-04, 1.33290221e-04, 1.33757083e-04, 1.33780138e-04,
1.34128524e-04, 1.34128524e-04, 1.34128524e-04, 1.34128524e-04,
1.34128524e-04, 1.34162023e-04, 1.35886015e-04, 1.35978572e-04,
1.36111885e-04, 1.36278319e-04, 1.36771450e-04, 1.37601396e-04,
1.37960767e-04, 1.37960767e-04, 1.37960767e-04, 1.37960767e-04,
1.37960767e-04, 1.38439798e-04, 1.39603157e-04, 1.39715010e-04,
1.39876889e-04, 1.40216202e-04, 1.40297066e-04, 1.40834950e-04,
1.41096239e-04, 1.41096239e-04, 1.41096239e-04, 1.41770534e-04,
1.41859026e-04, 1.43521278e-04, 1.43709133e-04, 1.43946347e-04,
1.44035744e-04, 1.44092492e-04, 1.45920042e-04, 1.47117676e-04,
1.47158152e-04, 1.47160380e-04, 1.47275337e-04, 1.47599795e-04,
1.48884864e-04, 1.50298168e-04, 1.50328648e-04, 1.50488905e-04,
1.50502655e-04, 1.50894589e-04, 1.51756844e-04, 1.52162611e-04,
1.52321114e-04, 1.52508435e-04, 1.52553712e-04, 1.52924766e-04,
1.53389729e-04, 1.53824420e-04, 1.54202181e-04, 1.56304816e-04,
1.56818649e-04, 1.57053552e-04, 1.57455224e-04, 1.57616257e-04,
1.57780482e-04, 1.59643088e-04, 1.59644744e-04, 1.60255712e-04,
1.60372339e-04, 1.61364257e-04, 1.61672774e-04, 1.61912290e-04,
1.63305833e-04, 1.64239009e-04, 1.64270438e-04, 1.66063887e-04,
1.67314646e-04, 1.68625919e-04, 1.68921261e-04, 1.68932563e-04,
1.70102542e-04, 1.72045455e-04, 1.72195276e-04, 1.72439875e-04,
1.72505968e-04, 1.72600267e-04, 1.73723533e-04, 1.73920712e-04,
1.74879297e-04, 1.74924370e-04, 1.76914396e-04, 1.77238979e-04,

1.77821907e-04, 1.78505159e-04, 1.79636416e-04, 1.79948827e-04,
1.81926287e-04, 1.83682009e-04, 1.83947690e-04, 1.86819455e-04,
1.88310570e-04, 1.89533129e-04, 1.89738323e-04, 1.90336920e-04,
1.91091512e-04, 1.91389018e-04, 1.91454601e-04, 1.91612177e-04,
1.94007329e-04, 1.94007329e-04, 1.94007329e-04, 1.94007329e-04,
1.94493563e-04, 1.95444420e-04, 1.95692847e-04, 1.96305933e-04,
1.96384845e-04, 1.96490696e-04, 1.96525507e-04, 1.97168930e-04,
1.97771140e-04, 1.98566551e-04, 2.00788503e-04, 2.02638135e-04,
2.05895748e-04, 2.07180666e-04, 2.07411472e-04, 2.09031466e-04,
2.09827310e-04, 2.10174607e-04, 2.10174607e-04, 2.10216255e-04,
2.11252425e-04, 2.11617132e-04, 2.12371139e-04, 2.12727335e-04,
2.13027656e-04, 2.13460021e-04, 2.14104343e-04, 2.14385321e-04,
2.15493739e-04, 2.16159728e-04, 2.16798579e-04, 2.17855406e-04,
2.20129827e-04, 2.20344191e-04, 2.21722662e-04, 2.21758956e-04,
2.21836457e-04, 2.22077987e-04, 2.24745144e-04, 2.26118887e-04,
2.26341884e-04, 2.27942294e-04, 2.29424780e-04, 2.29934612e-04,
2.31304659e-04, 2.34115242e-04, 2.35449765e-04, 2.36932984e-04,
2.45263587e-04, 2.49211085e-04, 2.51099048e-04, 2.51790376e-04,
2.52587122e-04, 2.53608377e-04, 2.55536575e-04, 2.57728811e-04,
2.58768560e-04, 2.59475786e-04, 2.61603026e-04, 2.62782414e-04,
2.63307904e-04, 2.63353003e-04, 2.65722523e-04, 2.68416725e-04,
2.70009644e-04, 2.71470911e-04, 2.71635343e-04, 2.71823599e-04,
2.74379317e-04, 2.78105022e-04, 2.78852248e-04, 2.81607962e-04,
2.83093230e-04, 2.85640884e-04, 2.88069181e-04, 2.92104296e-04,
2.92537642e-04, 2.95106704e-04, 2.95260486e-04, 2.99670849e-04,
2.99894713e-04, 3.03060885e-04, 3.03510209e-04, 3.05017693e-04,
3.05371327e-04, 3.08005434e-04, 3.21848858e-04, 3.24528642e-04,
3.25971249e-04, 3.28856288e-04, 3.29914719e-04, 3.36115063e-04,
3.36279371e-04, 3.39162452e-04, 3.39897532e-04, 3.40567501e-04,
3.42622690e-04, 3.43922084e-04, 3.44901919e-04, 3.45585705e-04,
3.53745557e-04, 3.54374938e-04, 3.57440356e-04, 3.68069316e-04,
3.68731506e-04, 3.82226127e-04, 3.86502143e-04, 3.90829083e-04,
3.97848021e-04, 4.00795115e-04, 4.01467027e-04, 4.08351701e-04,
4.14336320e-04, 4.16248036e-04, 4.16956698e-04, 4.25664482e-04,
4.29381773e-04, 4.30065577e-04, 4.41439298e-04, 4.43262185e-04,
4.45679457e-04, 4.56635672e-04, 4.57151816e-04, 4.59046973e-04,
4.64592146e-04, 4.70087732e-04, 4.74567838e-04, 4.78528593e-04,
4.82097805e-04, 4.82379708e-04, 4.87934902e-04, 4.91320955e-04,
5.13716150e-04, 5.20664881e-04, 5.30854359e-04, 5.35320896e-04,
5.57268608e-04, 5.79134374e-04, 5.83578196e-04, 5.85961232e-04,
5.89367112e-04, 6.00972199e-04, 6.18029260e-04, 6.36245190e-04,
6.39175621e-04, 6.39658698e-04, 6.50386475e-04, 6.52666755e-04,
6.53304236e-04, 6.57584454e-04, 6.72125043e-04, 6.75884070e-04,
6.96773565e-04, 7.02779693e-04, 8.02389918e-04, 8.09765155e-04,
8.27680639e-04, 8.28161684e-04, 8.91679944e-04, 9.06905177e-04,
9.11030947e-04, 1.06932727e-03, 1.16014793e-03, 1.22121842e-03,
1.28518223e-03, 1.31649255e-03, 1.33454696e-03, 1.60381340e-03,
1.66271922e-03, 1.83294958e-03, 1.83956000e-03, 1.85901075e-03,
1.91601907e-03, 2.06858567e-03, 2.09156784e-03, 2.15413734e-03,
2.40958731e-03, 2.61061954e-03, 2.95960729e-03, 3.19377357e-03,
4.10535944e-03, 4.51428291e-03, 5.02308256e-03, 1.00733591e-02,
1.04190956e-02, 1.11139914e-02, 2.66036110e-02, 9.56531268e-02,
1.01851708e-01])

```
In [100]: from sklearn.model_selection import GridSearchCV
clf_tree = DecisionTreeClassifier(random_state=2)
```

```
In [101]: grid = GridSearchCV(
    estimator=clf_tree,
    param_grid={
        'criterion': ['gini', 'entropy'],
        'max_depth': [12, 14, 16, 18, 20, 22, 24, 26, 28, 30],
        "ccp_alpha": [0.001, 0.005, 0.01, 0.02, 0.03, 0.04, 0.05]
    }
)
```

```
In [102]: grid.fit(X_train, Y_train)
```

```
Out[102]: GridSearchCV(estimator=DecisionTreeClassifier(random_state=2),
                        param_grid={'ccp_alpha': [0.001, 0.005, 0.01, 0.02, 0.03, 0.04,
                                                  0.05],
                                    'criterion': ['gini', 'entropy'],
                                    'max_depth': [12, 14, 16, 18, 20, 22, 24, 26, 28, 30]})
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



GridSearchCV

[?Documentation for GridSearchCV](#)iFitted

Parameters

	estimator estimator: estimator object This is assumed to implement the scikit-learn estimator interface. Either estimator needs to provide a ``score`` function, or ``scoring`` must be passed.	DecisionTreeC...andom_state=2)
	param_grid param_grid: dict or list of dictionaries Dictionary with parameters names (``str``) as keys and lists of parameter settings to try as values, or a list of such dictionaries, in which case the grids spanned by each dictionary in the list are explored. This enables searching over any sequence of parameter settings.	{'ccp_alpha': [0.001, 0.005, ...], 'criterion': ['gini', 'entropy'], 'max_depth': [12, 14, ...]}
	scoring scoring: str, callable, list, tuple or dict, default=None Strategy to evaluate the performance of the cross-validated model on the test set. If ``scoring`` represents a single score, one can use: - a single string (see :ref:`scoring_string_names`); - a callable (see :ref:`scoring_callable`) that returns a single value; - ``None``, the ``estimator``'s :ref:`default evaluation criterion <scoring_api_overview>` is used.	None

	If `scoring` represents multiple scores, one can use: - a list or tuple of unique strings; - a callable returning a dictionary where the keys are the metric names and the values are the metric scores; - a dictionary with metric names as keys and callables as values. See :ref:`multimetric_grid_search` for an example.	
	n_jobs n_jobs: int, default=None Number of jobs to run in parallel. ``None`` means 1 unless in a :obj:`joblib.parallel_backend` context. ``-1`` means using all processors. See :term:`Glossary <n_jobs>` for more details. .. versionchanged:: v0.20 `n_jobs` default changed from 1 to None	None
	refit refit: bool, str, or callable, default=True Refit an estimator using the best found parameters on the whole dataset. For multiple metric evaluation, this needs to be a `str` denoting the scorer that would be used to find the best parameters for refitting the estimator at the end. Where there are considerations other than maximum score in choosing a best estimator, ``refit`` can be set to a function which returns the selected ``best_index`` given ``cv_results``. In that case, the ``best_estimator_`` and ``best_params`` will be set according to the returned ``best_index`` while the ``best_score`` attribute will not be available. The refitted estimator is made available at the ``best_estimator_`` attribute and permits using ``predict`` directly on this ``GridSearchCV`` instance.	True

	Also for multiple metric evaluation, the attributes ``best_index``, ``best_score`` and ``best_params`` will only be available if ``refit`` is set and all of them will be determined w.r.t this specific scorer. See ``scoring`` parameter to know more about multiple metric evaluation. See <code>:ref:`sphx_glr_auto_examples_model_selection_plot_grid_search_digits.py`</code> to see how to design a custom selection strategy using a callable via ``refit``. See <code>:ref:`this example <sphx_glr_auto_examples_model_selection_plot_grid_search_refit_callable.py>`</code> for an example of how to use ``refit=callable`` to balance model complexity and cross-validated score. .. versionchanged:: 0.20 Support for callable added.	
	cv cv: int, cross-validation generator or an iterable, default=None Determines the cross-validation splitting strategy. Possible inputs for cv are: - None, to use the default 5-fold cross validation, - integer, to specify the number of folds in a ``(Stratified)KFold``, - :term:`CV splitter`, - an iterable yielding (train, test) splits as arrays of indices. For integer/None inputs, if the estimator is a classifier and ``y`` is either binary or multiclass, :class:`StratifiedKFold` is used. In all other cases, :class:`KFold` is used. These splitters are instantiated with ``shuffle=False`` so the splits will be the same across calls.	None

	Refer :ref:`User Guide <cross_validation>` for the various cross-validation strategies that can be used here. .. versionchanged:: 0.22 ``cv`` default value if None changed from 3-fold to 5-fold.	
	verbose verbose: int, default=0 Controls the verbosity of information printed during fitting, with higher values yielding more detailed logging. - 0 : no messages are printed; - >=1 : summary of the total number of fits; - >=2 : computation time for each fold and parameter candidate; - >=3 : fold indices and scores; - >=10 : parameter candidate indices and START messages before each fit.	0
	pre_dispatch pre_dispatch: int, or str, default='2*n_jobs' Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be: - None, in which case all the jobs are immediately created and spawned. Use this for lightweight and fast-running jobs, to avoid delays due to on-demand spawning of the jobs - An int, giving the exact number of total jobs that are spawned - A str, giving an expression as a function of n_jobs, as in '2*n_jobs'	'2*n_jobs'
	error_score error_score: 'raise' or numeric, default=np.nan Value to assign to the score if an error occurs in estimator fitting. If set to 'raise', the error is raised. If a numeric value is given, FitFailedWarning is raised. This	nan

	parameter does not affect the refit step, which will always raise the error.	
	return_train_score return_train_score: bool, default=False If ``False``, the ``cv_results_`` attribute will not include training scores. Computing training scores is used to get insights on how different parameter settings impact the overfitting/underfitting trade-off. However computing the scores on the training set can be computationally expensive and is not strictly required to select the parameters that yield the best generalization performance. .. versionadded:: 0.19 .. versionchanged:: 0.21 Default value was changed from ``True`` to ``False``	False

Fitted attributes

Name	Type	Value
best_estimator_ best_estimator_ : estimator Estimator that was chosen by the search, i.e. estimator which gave highest score (or smallest loss if specified) on the left out data. Not available if ``refit=False``. See ``refit`` parameter for more information on allowed values.	DecisionTreeClassifier	DecisionTreeC...andom_state=2)
best_index_ best_index_ : int The index (of the ``cv_results_`` arrays) which corresponds to the best candidate parameter setting. The dict at ``search.cv_results_[!params'][search.best_index_]`` gives the parameter setting for the best model, that gives the highest mean score	int64	np.int64(17)

(`search.best_score_`). For multi-metric evaluation, this is present only if ``refit`` is specified.		
best_params_ best_params_: dict Parameter setting that gave the best results on the hold out data. For multi-metric evaluation, this is present only if ``refit`` is specified.	dict	{'cc...ha': 0.001, 'cr...on': 'entropy', 'ma...th': 26}
best_score_ best_score_: float Mean cross-validated score of the best_estimator For multi-metric evaluation, this is present only if ``refit`` is specified. This attribute is not available if ``refit`` is a function.	float64	0.8848
classes_ classes_: ndarray of shape (n_classes,) The classes labels. This is present only if ``refit`` is specified and the underlying estimator is a classifier.	ndarray[int64](2,)	[0,1]
cv_results_ cv_results_: dict of numpy (masked) ndarrays A dict with keys as column headers and values as columns, that can be imported into a pandas ``DataFrame``. For instance the below given table <pre> +-----+-----+-----+ ----+-----+-----+-----+ ----+ param_kernel param_gamma param_degree split0_test_score ... rank_t... +=====+=====+ ==+=====+=====+ =====+=====+=====+ 'poly' -- 2 0.80 ... 2 +-----+-----+-----+ ----+-----+-----+ ----+ 'poly' -- 3 0.70 ... 4 </pre>	dict	{'me...me': array([0.1127..., 0.12837667]), 'me...me': array([0.0020..., 0.00301313]), 'me...re': array([0.8735..., 0.78659194]), 'pa...ha': masked_array(...l_value=1e+20), ...}

```

+-----+-----+-----
+-----+-----+-----
+-----+
|'rbf'| 0.1 | -- | 0.80 |...| 3 |
+-----+-----+-----
+-----+
|'rbf'| 0.2 | -- | 0.93 |...| 1 |
+-----+-----+-----
+-----+
|'rbf'| 0.2 | -- | 0.93 |...| 1 |
+-----+-----+-----
+-----+

```

will be represented by a
`cv\_results\_` dict of::

```

{
  'param_kernel':
  masked_array(data = ['poly',
  'poly', 'rbf', 'rbf'],
  mask = [False False False
  False]...),
  'param_gamma':
  masked_array(data = [-- -- 0.1
  0.2],
  mask = [ True True False
  False]...),
  'param_degree':
  masked_array(data = [2.0 3.0 -- -
  -]),
  mask = [False False True True]...),
  'split0_test_score': [0.80, 0.70,
  0.80, 0.93],
  'split1_test_score': [0.82, 0.50,
  0.70, 0.78],
  'mean_test_score': [0.81, 0.60,
  0.75, 0.85],
  'std_test_score': [0.01, 0.10, 0.05,
  0.08],
  'rank_test_score': [2, 4, 3, 1],
  'split0_train_score': [0.80, 0.92,
  0.70, 0.93],
  'split1_train_score': [0.82, 0.55,
  0.70, 0.87],
  'mean_train_score': [0.81, 0.74,
  0.70, 0.90],
  'std_train_score': [0.01, 0.19,
  0.00, 0.03],
  'mean_fit_time': [0.73, 0.63, 0.43,
  0.49],
  'std_fit_time': [0.01, 0.02, 0.01,
  0.01],
  'mean_score_time': [0.01, 0.06,
  0.04, 0.04],
  'std_score_time': [0.00, 0.00,
  0.00, 0.01],
  'params': [{ 'kernel': 'poly',
  'degree': 2}, ...],
}

```

[For an example of visualization
and interpretation of GridSearch](#)

results , see :ref:`sphx_glr_auto_examples_model_selection_plot_grid_search_stats.py` . NOTE The key ``'params'`` is used to store a list of parameter settings dicts for all the parameter candidates. The ``mean_fit_time``, ``std_fit_time``, ``mean_score_time`` and ``std_score_time`` are all in seconds. For multi-metric evaluation, the scores for all the scorers are available in the ``cv_results`` dict at the keys ending with that scorer's name (``'_<scorer_name>'``) instead of ``'_score'`` shown above. (``'split0_test_precision'``, ``'mean_train_precision'`` etc.)		
feature_names_in_ feature_names_in_ : ndarray of shape (n_features_in_,) Names of features seen during :term:`fit`. Only defined if ``best_estimator_`` is defined (see the documentation for the ``refit`` parameter for more details) and that ``best_estimator_`` exposes ``feature_names_in_`` when fit. .. versionadded:: 1.0	ndarray(object)(32,)	['Month','WeekOfMonth','DayOfWeek',...,'NumberOfCars','Year','BasePolicy']
multimetric_ multimetric_ : bool Whether or not the scorers compute several metrics.	bool	False
n_features_in_ n_features_in_ : int Number of features seen during :term:`fit`. Only defined if ``best_estimator_`` is defined (see the documentation for the ``refit`` parameter for more details) and that ``best_estimator_`` exposes ``n_features_in_`` when fit. .. versionadded:: 0.24	int	32
n_splits_ n_splits_ : int	int	5

The number of cross-validation splits (folds/iterations).		
refit_time_ refit_time_: float Seconds used for refitting the best model on the whole dataset. This is present only if ``refit`` is not False. .. versionadded:: 0.20	float	0.1606
scorer_ scorer_: function or a dict Scorer function used on the held out data to choose the best parameters for the model. For multi-metric evaluation, this attribute holds the validated ``scoring`` dict which maps the scorer key to the scorer callable.	_PassthroughScorer	DecisionTreeClassifier.score



best_estimator_: DecisionTreeClassifier

DecisionTreeClassifier(ccp_alpha=0.001, criterion='entropy', max_depth=26, random_state=2)



DecisionTreeClassifier

[?Documentation for DecisionTreeClassifier](#)

Parameters

	criterion criterion: {"gini", "entropy", "log_loss"}, default="gini" The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:`tree_mathematical_formulation`.	'entropy'
	max_depth max_depth: int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.	26
	random_state random_state: int, RandomState instance or None,	2

	default=None Controls the randomness of the estimator. The features are always randomly permuted at each split, even if ``splitter`` is set to ``"best"``. When ``max_features < n_features``, the algorithm will select ``max_features`` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if ``max_features=n_features``. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, ``random_state`` has to be fixed to an integer. See :term:`Glossary` <random_state>` for details.	
	ccp_alpha ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ``ccp_alpha`` will be chosen. By default, no pruning is performed. See :ref:`minimal_cost_complexity_pruning` for details. See :ref:`sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py` for an example of such pruning. .. versionadded:: 0.22	0.001
	splitter splitter: {"best", "random"}, default="best" The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split among considered features for this split.	'best'
	min_samples_split min_samples_split: int or float,	2

	<u>default=2</u> <u>The minimum number of samples required to split an internal node:</u> - If int, then consider <u>`min_samples_split` as the minimum number.</u> - If float, then <u>`min_samples_split` is a fraction and $\text{ceil}(\text{min_samples_split} * \text{n_samples})$ are the minimum number of samples for each split.</u> <u>.. versionchanged:: 0.18</u> <u>Added float values for fractions.</u>	
	<u>min_samples_leaf</u> <u>min_samples_leaf: int or float,</u> <u>default=1</u> <u>The minimum number of samples required to be at a leaf node.</u> <u>A split point at any depth will only be considered if it leaves at least ``min_samples_leaf`` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.</u> - If int, then consider <u>`min_samples_leaf` as the minimum number.</u> - If float, then <u>`min_samples_leaf` is a fraction and $\text{ceil}(\text{min_samples_leaf} * \text{n_samples})$ are the minimum number of samples for each node.</u> <u>.. versionchanged:: 0.18</u> <u>Added float values for fractions.</u>	1
	<u>min_weight_fraction_leaf</u> <u>min_weight_fraction_leaf: float,</u> <u>default=0.0</u> <u>The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when <u>sample_weight</u> is not provided.</u>	0.0
	<u>max_features</u> <u>max_features: int,</u> <u>float or {"sqrt", "log2"},</u> <u>default=None</u> <u>The number of features to consider when looking for the best split:</u>	None

	- If int, then consider `max_features` features at each split. - If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split. - If "sqrt", then `max_features=sqrt(n_features)`. - If "log2", then `max_features=log2(n_features)`. - If None, then `max_features=n_features`. Note: splitting may inspect more than ``max_features`` features if needed to find a valid split.	
	<u>max_leaf_nodes</u> <u>max_leaf_nodes</u>: int, default=None Grow a tree with ``max_leaf_nodes`` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	None
	<u>min_impurity_decrease</u> <u>min_impurity_decrease</u>: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. The weighted impurity decrease equation is the following:: $N_t / N * (impurity - N_{t_R} / N_t * right_impurity - N_{t_L} / N_t * left_impurity)$ where ``N`` is the total number of samples, ``N_t`` is the number of samples at the current node, ``N_t_L`` is the number of samples in the left child, and ``N_t_R`` is the number of samples in the right child. ``N``, ``N_t``, ``N_t_R`` and ``N_t_L`` all refer to the weighted sum, if ``sample_weight`` is passed.	0.0

	.. versionadded:: 0.19	
	class_weight class_weight: dict, list of dict or "balanced", default=None Weights associated with classes in the form ``{class_label: weight}``. If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as ``n_samples / (n_classes * np.bincount(y))`` For multi-output, the weights of each column of y will be multiplied. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.	None
	monotonic_cst monotonic_cst: array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for:	None

	- multiclass classifications (i.e. when 'n_classes' > 2)., - multioutput classifications (i.e. when 'n_outputs' > 1)., The constraints hold over the probability of the positive class. Read more in the :ref:'User Guide <monotonic_cst_gbd>'. .. versionadded:: 1.4	
--	--	--

Fitted attributes

Name	Type	Value
classes_: ndarray of shape (n_classes,) or list of ndarray The classes labels (single output problem)., or a list of arrays of class labels (multi-output problem).	ndarray[int64](2,)	[0,1]
feature_importances_: ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance [4]. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:'sklearn.inspection.permutation_importance' as an alternative.	ndarray[float64](32,)	[0. ,0. ,0.01,...,0. ,0. ,0.37]
feature_names_in_: ndarray of shape (n_features_in_,) Names of features seen during :term:'fit'. Defined only when 'X' has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](32,)	['Month','WeekOfMonth','DayOfWeek',...,'NumberOfCars','Year','BasePolicy']
max_features_ : int	int	32

The inferred value of max_features.		
n_classes_n_classes_ : int or list of int The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).	int64	np.int64(2)
n_features_in_n_features_in_ : int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	32
n_outputs_n_outputs_ : int The number of outputs when ``fit`` is performed.	int	1
tree_tree_ : Tree instance The underlying Tree object. Please refer to ``help(sklearn.tree._tree.Tree)`` for attributes of Tree object and :ref:`sphx_glr_auto_examples_tree_plot_unveil_tree_structure.py` for basic usage of these attributes.	Tree	<sklearn.tree...001D58DC7E250>

In [103]: `print(grid.best_params_)`

```
{'ccp_alpha': 0.001, 'criterion': 'entropy', 'max_depth': 26}
```

```
In [104]: dt_clf1 = DecisionTreeClassifier(criterion = 'entropy', max_depth = 26,  
      ccp_alpha = 0.001)  
dt_clf1.fit(X_train, Y_train)
```

Out[104]: DecisionTreeClassifier(ccp_alpha=0.001, criterion='entropy', max_depth=26)

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



DecisionTreeClassifier

[?Documentation for DecisionTreeClassifier](#)Fitted

Parameters

	criterion criterion: {"gini", "entropy", "log_loss"}, default="gini" The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:`tree_mathematical_formulation`.	'entropy'
	max_depth max_depth: int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.	26
	ccp_alpha ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ``ccp_alpha`` will be chosen. By default, no pruning is performed. See :ref:`minimal_cost_complexity_pruning` for details. See :ref:`sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py` for an example of such pruning. .. versionadded:: 0.22	0.001
	splitter splitter: {"best", "random"}, default="best" The strategy used to choose the	'best'

	split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split among considered features for this split.	
	min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider `min_samples_split` as the minimum number. - If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	2
	min_samples_leaf min_samples_leaf: int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least ``min_samples_leaf`` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression. - If int, then consider `min_samples_leaf` as the minimum number. - If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node. .. versionchanged:: 0.18 Added float values for fractions.	1
	min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have	0.0

	equal weight when sample_weight is not provided.	
	max_features <code>max_features: int, float or {"sqrt", "log2"}, default=None</code> The number of features to consider when looking for the best split: - If int, then consider `max_features` features at each split. - If float, then `max_features` is a fraction and <code>`max(1, int(max_features * n_features_in_))`</code> features are considered at each split. - If "sqrt", then <code>`max_features=sqrt(n_features)`</code>. - If "log2", then <code>`max_features=log2(n_features)`</code>. - If None, then <code>`max_features=n_features`</code>. Note: splitting may inspect more than ``max_features`` features if needed to find a valid split.	None
	random_state <code>random_state: int, RandomState instance or None, default=None</code> Controls the randomness of the estimator. The features are always randomly permuted at each split, even if ``splitter`` is set to ``"best"``. When ``max_features < n_features``, the algorithm will select ``max_features`` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if <code>`max_features=n_features`</code>. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, ``random_state`` has to be fixed to an integer. See :term:`Glossary` <code><random_state></code> for details.	None

	max_leaf_nodes max_leaf_nodes: int, default=None Grow a tree with ``max_leaf_nodes`` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	None
	min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. The weighted impurity decrease equation is the following:: $\frac{N_t}{N} * (\text{impurity} - \frac{N_{t_R}}{N_t} * \text{right_impurity} - \frac{N_{t_L}}{N_t} * \text{left_impurity})$ where ``N`` is the total number of samples, ``N_t`` is the number of samples at the current node, ``N_t_L`` is the number of samples in the left child, and ``N_t_R`` is the number of samples in the right child. ``N``, ``N_t``, ``N_t_R`` and ``N_t_L`` all refer to the weighted sum, if ``sample_weight`` is passed. .. versionadded:: 0.19	0.0
	class_weight class_weight: dict, list of dict or "balanced", default=None Weights associated with classes in the form ``{class_label: weight}``. If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every	None

	column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of <code>y</code> to automatically adjust weights inversely proportional to class frequencies in the input data as <code>``n_samples / (n_classes * np.bincount(y))``</code> For multi-output, the weights of each column of <code>y</code> will be multiplied. Note that these weights will be multiplied with <code>sample_weight</code> (passed through the fit method) if <code>sample_weight</code> is specified.	
	monotonic_cst monotonic_cst: array-like of int of shape (n_features),, default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease If <code>monotonic_cst</code> is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when <code>``n_classes > 2``</code>), - multioutput classifications (i.e. when <code>``n_outputs_ > 1``</code>). The constraints hold over the probability of the positive class. Read more in the :ref:`User Guide <monotonic_cst_gbd>`. <code>.. versionadded:: 1.4</code>	None

Fitted attributes

Name	Type	Value
classes_ classes_ : ndarray of shape (n_classes,) or list of ndarray	ndarray[int64](2,)	[0,1]

The classes labels (single output problem). , or a list of arrays of class labels (multi-output problem).		
feature_importances_ feature_importances_ : ndarray of shape (n_features,)	ndarray[float64](32,)	[0. ,0. ,0.01,...,0. ,0. ,0.37]
The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance [4].		
Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:'sklearn.inspection.permutation_importance' as an alternative.		
feature_names_in_ feature_names_in_ : ndarray of shape (n_features_in_,)	ndarray[object](32,)	['Month','WeekOfMonth','DayOfWeek',...,'NumberOfCars','Year','BasePolicy']
Names of features seen during :term:'fit'. Defined only when 'X' has feature names that are all strings. .. versionadded:: 1.0		
max_features_ max_features_: int	int	32
The inferred value of max_features.		
n_classes_ n_classes_: int or list of int	int64	np.int64(2)
The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).		
n_features_in_ n_features_in_: int	int	32
Number of features seen during :term:'fit'. .. versionadded:: 0.24		


```
In [87]: rf_clf.fit(X_train, Y_train)
```

```
Out[87]: RandomForestClassifier(max_depth=15, max_features=10, n_estimators=50,
                                random_state=2)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



RandomForestClassifier

[?Documentation for RandomForestClassifier](#)iFitted

Parameters

	n_estimators n_estimators: int, default=100 The number of trees in the forest. .. versionchanged:: 0.22 The default value of ``n_estimators`` changed from 10 to 100 in 0.22.	50
	max_depth max_depth: int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.	15
	max_features max_features: {"sqrt", "log2", None}, int or float, default="sqrt" The number of features to consider when looking for the best split: - If int, then consider ``max_features`` features at each split. - If float, then ``max_features`` is a fraction and ``max(1, int(max_features * n_features_in_))`` features are considered at each split. - If "sqrt", then ``max_features=sqrt(n_features)``. - If "log2", then ``max_features=log2(n_features)``. - If None, then ``max_features=n_features``. .. versionchanged:: 1.1	10

	The default of `max_features` changed from `"auto"` to `"sqrt"`. Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.	
	random_state random_state: int, RandomState instance or None, default=None Controls both the randomness of the bootstrapping of the samples used when building trees (if `bootstrap=True`) and the sampling of the features to consider when looking for the best split at each node (if `max_features < n_features`). See :term:`Glossary` <random_state>` for details.	2
	criterion criterion: {"gini", "entropy", "log_loss"}, default="gini" The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:`tree_mathematical_formulation`. Note: This parameter is tree-specific.	'gini'
	min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider `min_samples_split` as the minimum number. - If float, then `min_samples_split` is a fraction and $\text{ceil}(\text{min_samples_split} * \text{n_samples})$ are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	2

	<u>min_samples_leaf</u> <u>min_samples_leaf: int or float, default=1</u> <u>The minimum number of samples required to be at a leaf node.</u> <u>A split point at any depth will only be considered if it leaves at least ``min_samples_leaf`` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.</u> <u>- If int, then consider ``min_samples_leaf`` as the minimum number.</u> <u>- If float, then ``min_samples_leaf`` is a fraction and ``ceil(min_samples_leaf * n_samples)`` are the minimum number of samples for each node.</u> <u>.. versionchanged:: 0.18</u> <u>Added float values for fractions.</u>	1
	<u>min_weight_fraction_leaf</u> <u>min_weight_fraction_leaf: float, default=0.0</u> <u>The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.</u>	0.0
	<u>max_leaf_nodes</u> <u>max_leaf_nodes: int, default=None</u> <u>Grow trees with ``max_leaf_nodes`` in best-first fashion.</u> <u>Best nodes are defined as relative reduction in impurity.</u> <u>If None then unlimited number of leaf nodes.</u>	None
	<u>min_impurity_decrease</u> <u>min_impurity_decrease: float, default=0.0</u> <u>A node will be split if this split induces a decrease of the impurity greater than or equal to this value.</u> <u>The weighted impurity decrease equation is the following::</u>	0.0

	N_t / N * (impurity - N_t_R / N_t * right_impurity - N_t_L / N_t * left_impurity) where ``N`` is the total number of samples, ``N_t`` is the number of samples at the current node, ``N_t_L`` is the number of samples in the left child, and ``N_t_R`` is the number of samples in the right child. ``N``, ``N_t``, ``N_t_R`` and ``N_t_L`` all refer to the weighted sum, if ``sample_weight`` is passed. .. versionadded:: 0.19	
	bootstrap bootstrap: bool, default=True Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.	True
	oob_score oob_score: bool or callable, default=False Whether to use out-of-bag samples to estimate the generalization score. By default, :func:`~sklearn.metrics.accuracy_score` is used. Provide a callable with signature <code>`metric(y_true, y_pred)`</code> to use a custom metric. Only available if <code>`bootstrap=True`</code>. For an illustration of out-of-bag (OOB) error estimation, see the example :ref:`sphx_glr_auto_examples_ensemble_plot_ensemble_oob.py`.	False
	n_jobs n_jobs: int, default=None The number of jobs to run in parallel. :meth:`fit`, :meth:`predict`, :meth:`decision_path` and :meth:`apply` are all parallelized over the trees. ``None`` means 1 unless in a :obj:`joblib.parallel_backend` context. ``-1`` means using all processors. See :term:`Glossary` <n_jobs> for more details.	None

	verbose verbose: int, default=0 Controls the verbosity when fitting and predicting.	0
	warm_start warm_start: bool, default=False When set to ``True``, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest. See :term:`Glossary` <warm_start>` and :ref:`tree_ensemble_warm_start` for details.	False
	class_weight class_weight: {"balanced", "balanced_subsample"}, dict or list of dicts, default=None Weights associated with classes in the form ``{class_label: weight}``. If not given, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as ``n_samples / (n_classes * np.bincount(y))`` The "balanced_subsample" mode is the same as "balanced" except that weights are computed based on the bootstrap sample for every tree grown. For multi-output, the weights of	None

	each column of y will be multiplied. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.	
	ccp_alpha ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ``ccp_alpha`` will be chosen. By default, no pruning is performed. See :ref:`minimal_cost_complexity_pruning` for details. See :ref:`sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py` for an example of such pruning. .. versionadded:: 0.22	0.0
	max_samples max_samples: int or float, default=None If bootstrap is True, the number of samples to draw from X to train each base estimator. - If None (default), then draw ``X.shape[0]`` samples irrespective of ``sample_weight``. - If int, then draw ``max_samples`` samples. - If float, then draw ``max_samples * X.shape[0]`` unweighted samples or ``max_samples * sample_weight.sum()`` weighted samples. .. versionadded:: 0.22 .. versionchanged:: 1.9 Float ``max_samples`` is relative to ``sample_weight.sum()`` instead of ``X.shape[0]`` for weighted samples.	None
	monotonic_cst monotonic_cst: array-like of int of shape (n_features,), default=None Indicates the monotonicity	None

	constraint to enforce on each feature. - 1: monotonically increasing - 0: no constraint - -1: monotonically decreasing If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when 'n_classes' > 2), - multioutput classifications (i.e. when 'n_outputs' > 1). The constraints hold over the probability of the positive class. Read more in the :ref:`User Guide <monotonic_cst_gbdtd>`. .. versionadded:: 1.4	
--	---	--

Fitted attributes

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) or a list of such arrays The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).	ndarray[int64](2,)	[0,1]
estimator_ estimator_: :class:`~sklearn.tree.DecisionTreeClassifier` The child estimator template used to create the collection of fitted sub-estimators. .. versionadded:: 1.2 `base_estimator_` was renamed to `estimator_`.	DecisionTreeClassifier	DecisionTreeClassifier()
estimators_ estimators_: list of DecisionTreeClassifier The collection of fitted sub-estimators.	list	[DecisionTreeC...te=1872583848), , DecisionTreeC...ate=794921487), DecisionTreeC...ate=111352301), DecisionTreeC...te=1853453896), ...]
estimators_samples_ estimators_samples_: list of arrays The subset of drawn samples (i.e., the in-bag samples) for each base	list	[array([5939,..., dtype=int32), array([9825,..., dtype=int32), array([18448,..., dtype=int32), array([5721,..., dtype=int32), ...]

estimator. Each subset is defined by an array of the indices selected. .. versionadded:: 1.4		
feature_importances_ feature_importances_ : ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:'sklearn.inspection.permutation_importance' as an alternative.	ndarray[float64](32,)	[0.02,0.01,0.02,...,0. ,0.01,0.19]
feature_names_in_ feature_names_in_ : ndarray of shape (n_features_in_,) Names of features seen during :term:'fit'. Defined only when 'X' has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](32,)	['Month','WeekOfMonth','DayOfWeek',...,'NumberOfCars','Year','BasePolicy']
n_classes_ n_classes_ : int or list The number of classes (single output problem), or a list containing the number of classes for each output (multi-output problem).	int	2
n_features_in_ n_features_in_ : int Number of features seen during :term:'fit'. .. versionadded:: 0.24	int	32
n_outputs_ n_outputs_ : int The number of outputs when ``fit`` is performed.	int	1

```
In [88]: rf_clf.score(X_test, Y_test)
```

```
Out[88]: 0.9387825487152958
```

1. Evaluating Model Performance And Confusion Matrix

1. Logistic Regression

```
In [77]: from sklearn.metrics import confusion_matrix, accuracy_score,  
         classification_report
```

```
In [124]: y_pred_Logistic = logit_model.predict(X_test)
```

```
In [128]: print(classification_report(Y_test, y_pred_Logistic))
```

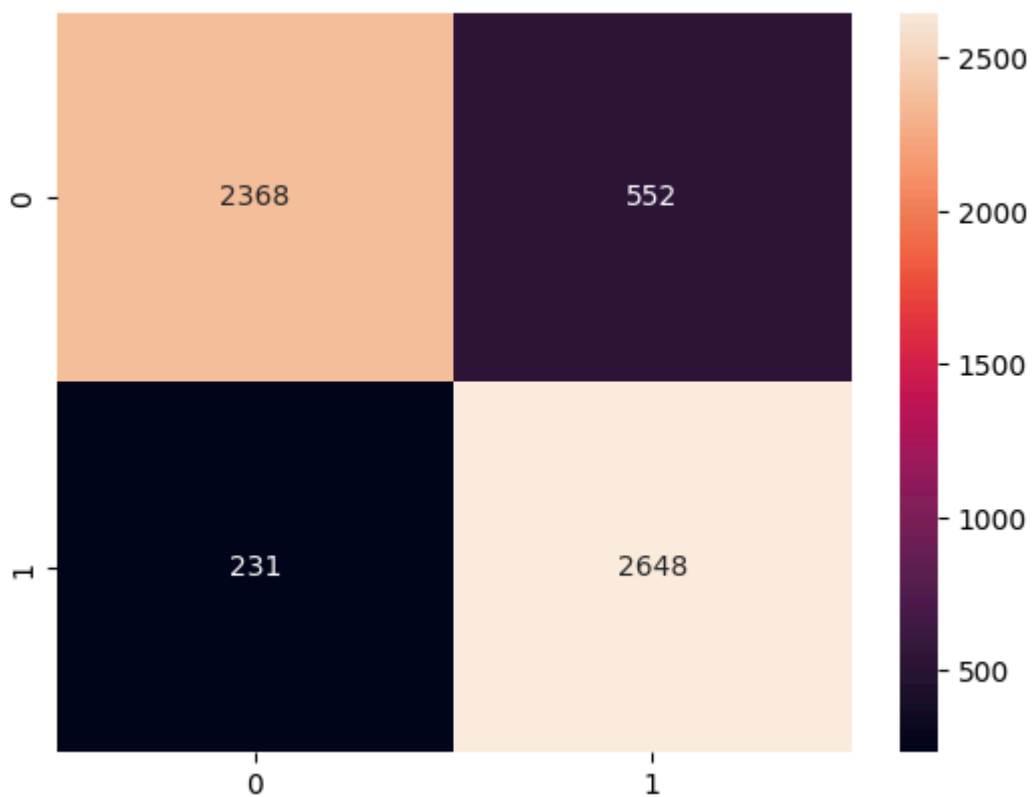
	precision	recall	f1-score	support
0	0.91	0.81	0.86	2920
1	0.83	0.92	0.87	2879
accuracy			0.86	5799
macro avg	0.87	0.87	0.86	5799
weighted avg	0.87	0.86	0.86	5799

```
In [ ]: # This is matrix between Actual Value and Predicted Value  
        confusion_matrix(Y_test, y_pred_Logistic)
```

```
Out[126]: array([[2368,  552],  
                [ 231, 2648]])
```

```
In [ ]: sns.heatmap(confusion_matrix(Y_test, y_pred_Logistic), annot = True, fmt = '0.0f')
```

```
Out[109]: <Axes: >
```



1. Decision Tree

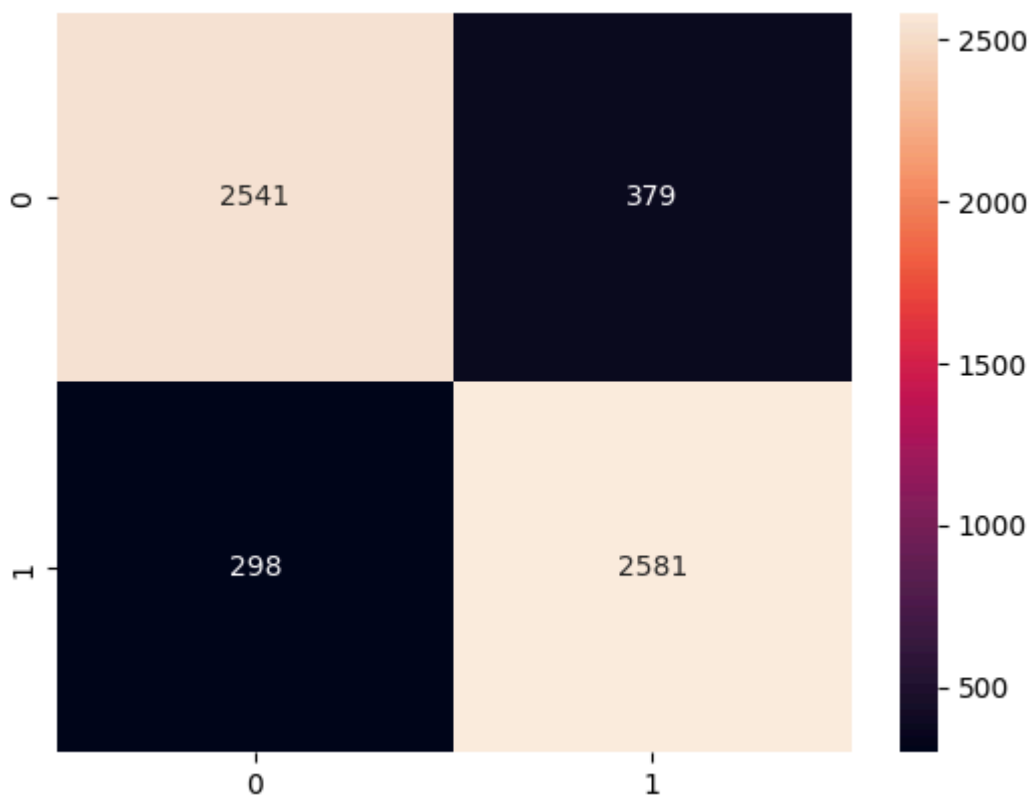
```
In [129]: ypred_DTree = dt_clf1.predict(X_test)
```

```
In [130]: print(classification_report(Y_test, ypred_DTree))
```

	precision	recall	f1-score	support
0	0.90	0.87	0.88	2920
1	0.87	0.90	0.88	2879
accuracy			0.88	5799
macro avg	0.88	0.88	0.88	5799
weighted avg	0.88	0.88	0.88	5799

```
In [131]: sns.heatmap(confusion_matrix(Y_test, ypred_DTree), annot = True, fmt =
'0.0f')
```

```
Out[131]: <Axes: >
```



1. Random Forest

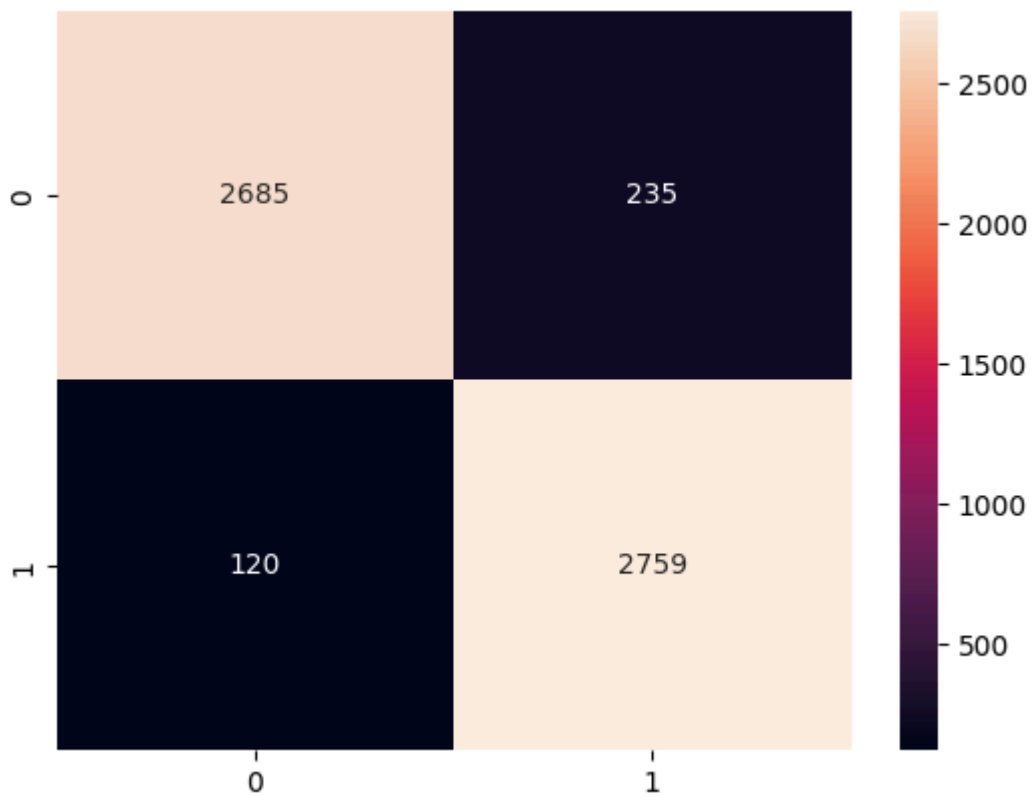
```
In [132]: ypred_RandomForest = rf_clf.predict(X_test)
```

```
In [133]: print(classification_report(Y_test, ypred_RandomForest))
```

	precision	recall	f1-score	support
0	0.96	0.92	0.94	2920
1	0.92	0.96	0.94	2879
accuracy			0.94	5799
macro avg	0.94	0.94	0.94	5799
weighted avg	0.94	0.94	0.94	5799

```
In [134]: sns.heatmap(confusion_matrix(Y_test, ypred_RandomForest), annot = True, fmt  
= '0.0f')
```

```
Out[134]: <Axes: >
```



1. Most Important Features Using Random Forest

```

In [120]: feature_importance = pd.DataFrame({
            'Feature': X.columns,
            'Importance': rf_clf.feature_importances_
        })

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

feature_importance.head(20)

```

Out[120]:

	Feature	Importance
31	BasePolicy	0.186256
11	Fault	0.172237
3	Make	0.064288
13	VehicleCategory	0.061034
22	AgeOfVehicle	0.059003
12	PolicyType	0.054921
15	PolicyNumber	0.040184
9	MaritalStatus	0.037386
10	Age	0.036387
18	DriverRating	0.032029
21	PastNumberOfClaims	0.027856
5	DayOfWeekClaimed	0.027657
16	RepNumber	0.026011
23	AgeOfPolicyHolder	0.021863
14	VehiclePrice	0.020029
2	DayOfWeek	0.018305
6	MonthClaimed	0.017012
0	Month	0.016705
27	NumberOfSuppliments	0.014902
7	WeekOfMonthClaimed	0.014491

```
In [121]: print("Top 10 Important Features:")

for i, row in feature_importance.head(10).iterrows():
    print(
        f"{row['Feature']}: {row['Importance']:.4f}"
    )
```

```
Top 10 Important Features:
BasePolicy: 0.1863
Fault: 0.1722
Make: 0.0643
VehicleCategory: 0.0610
AgeOfVehicle: 0.0590
PolicyType: 0.0549
PolicyNumber: 0.0402
MaritalStatus: 0.0374
Age: 0.0364
DriverRating: 0.0320
```

1. Comparing The Model Performance

```

In [137]: results = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'Decision Tree',
        'Random Forest'
    ],
    'Accuracy': [
        0.86,
        0.88,
        0.94
    ],
    'Precision': [
        0.87,
        0.88,
        0.94
    ],
    'Recall': [
        0.87,
        0.88,
        0.94
    ],
    'F1 Score': [
        0.86,
        0.88,
        0.94
    ]
})

results

```

```
Out[137]:
```

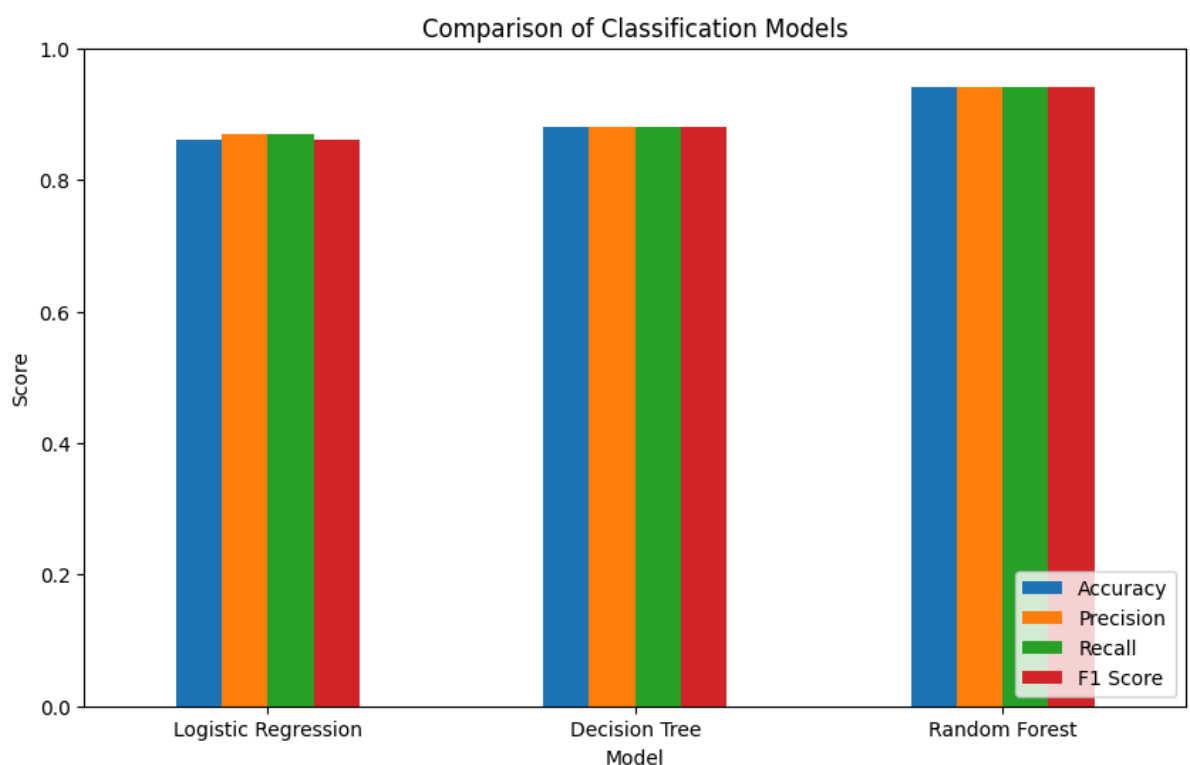
	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.86	0.87	0.87	0.86
1	Decision Tree	0.88	0.88	0.88	0.88
2	Random Forest	0.94	0.94	0.94	0.94

```
In [138]: results.set_index('Model').plot(
            kind='bar',
            figsize=(10, 6)
        )

plt.title('Comparison of Classification Models')
plt.xlabel('Model')
plt.ylabel('Score')
plt.ylim(0, 1)

plt.xticks(rotation=0)
plt.legend(loc='lower right')

plt.show()
```



Comparison of Classification Models

The three classification models — Logistic Regression, Decision Tree, and Random Forest — were evaluated using Accuracy, Precision, Recall, and F1 Score.

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	86.50%	87%	87%	86%
Decision Tree	88.33%	88%	88%	88%
Random Forest	93.88%	94%	94%	94%

The results show that Random Forest achieved the highest overall performance among the three evaluated models. It achieved an accuracy of approximately 93.88%, compared with 88.33% for Decision Tree and 86.50% for Logistic Regression.

Random Forest also achieved the highest Precision, Recall, and F1 Score. Therefore, based on the evaluation results obtained on the test dataset, Random Forest provides the strongest predictive performance among the three models evaluated in this study.

17. Key Insights

The following key insights were obtained from the analysis:

1. **Random Forest achieved the highest overall performance.** It achieved an accuracy of 93.88%, which was higher than Decision Tree (88.33%) and Logistic Regression (86.50%).
2. **Random Forest provided the highest fraud detection capability.** For the fraud class, Random Forest achieved a recall of 96%, compared with 92% for Logistic Regression and 90% for Decision Tree.
3. **Random Forest achieved higher fraud precision.** The fraud precision of Random Forest was 92%, meaning that most claims predicted as fraudulent were actually fraudulent.
4. **The F1 Score supports the Random Forest result.** Random Forest achieved an F1 Score of 94% for the fraud class, indicating a strong balance between precision and recall.
5. **Decision Tree performed better than Logistic Regression.** Decision Tree achieved 88.33% accuracy compared with 86.50% for Logistic Regression.
6. **The results demonstrate the importance of handling class imbalance.** SMOTE was applied to the training data to balance the target classes before training the classification models.
7. **Fraud detection should focus on both financial protection and operational efficiency.** Identifying fraudulent claims can help reduce potential financial losses, while maintaining good precision can help avoid sending excessive numbers of legitimate claims for manual investigation.

Recommended Model for Fraud Detection

Based on the evaluation results obtained from the test dataset, **Random Forest is the recommended model for this fraud detection assignment.**

Random Forest achieved:

- **93.88% Accuracy**
- **94% Precision**
- **94% Recall**
- **94% F1 Score**

For the fraud class specifically, Random Forest achieved:

- **92% Precision**
- **96% Recall**
- **94% F1 Score**

The high recall for the fraud class is particularly relevant to the insurance business because failing to identify an actual fraudulent claim may result in financial losses. The high precision is also valuable because it can help reduce the number of legitimate claims incorrectly sent for fraud investigation.

Therefore, based on the experimental results, Random Forest provides the most effective combination of fraud detection capability and prediction accuracy among the three models tested.

However, the model should be further validated on new and unseen insurance claim data before being deployed in a real-world insurance environment. In a production system, the decision threshold could also be adjusted according to the company's tolerance for missed fraud cases versus unnecessary investigations.

Final Conclusion

The objective of this project was to develop classification models for detecting fraudulent insurance claims using the Fraud Oracle dataset.

The dataset was cleaned and analyzed through exploratory data analysis. Categorical variables were encoded, the data was divided into training and testing sets, and SMOTE was applied to the training data to address class imbalance. Three classification algorithms — Logistic Regression, Decision Tree, and Random Forest — were then trained and evaluated.

Among the three models, Random Forest produced the strongest results, achieving approximately 93.88% accuracy and 94% for precision, recall, and F1 Score. For the fraud class, it achieved 92% precision and 96% recall.

From a business perspective, these results indicate that the Random Forest model can effectively identify a high proportion of potentially fraudulent claims while maintaining a relatively high level of precision. This can support insurance companies in prioritizing suspicious claims for further investigation, potentially reducing financial losses associated with fraudulent claims and improving the efficiency of the claims-review process.

Therefore, based on the results of this experiment, **Random Forest is recommended as the model for fraud detection among the three models evaluated.** Further validation using new real-world claims data would be required before production deployment.